

claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_2a64bc88-2a8\agent-aa73d5b35027200d4.jsonl`
- SHA-256: `209b53a195c5452b63bc4936bfee13c7ae83d41466b1e479bdc73e25f1eb394d`
- Source modified: `2026-06-10T06:06:42+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_2a64bc88-2a8`
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`
```

Transcript

1. user (2026-06-10T05:59:28.043Z)

CONTEXT ? two sibling repos on a Windows machine:

- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).

- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).

Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources. ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.

QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.

TASK: deep TESTING audit of the INDEXER repo. Steps:

1. Inventory: Glob tests/** and map every test file to the backend module(s) it covers. Read pytest.ini, run_all_tests.py, TEST_RUN_SUMMARY.md, and any .github/workflows/ (does CI exist at all? check with Glob).
 2. RUN the suite: from C:/Users/avidu/Projects/badminton-highlight-indexer run "python run_all_tests.py" (offline/mocked, safe). Capture the real pass/fail/skip counts and runtime. If it fails, report exactly what failed.
 3. Coverage-gap analysis: which backend modules have NO test file at all (Glob backend/**/*.py vs tests)? Which critical paths are untested ? FastAPI endpoints (/api/process, /api/compile, /api/query), stitcher/compiler ffmpeg interaction, WSL runners (wasb_runner/tracknet_runner), database migration ladder, config loader edge cases, reporting/harvest+spec.yaml rule firing?
 4. Test QUALITY: sample ~10 test files across subsystems and judge assertion depth (smoke vs behavioral), mocking strategy (are mocks faithful to real contracts? e.g. provider analyze_video tuple shape, ffmpeg output), fixture realism, the faq-resolution golden test, regression-gate tests.
 5. Test infra: marker/selection hygiene, runtime, flakiness risk (tmp paths, ordering, global registry mutation between tests), whether tests would catch the known copy-paste drift between wasb_hybrid/tracknet_hybrid.
- Report real numbers (file counts, test counts, runtime). Severity-rate gaps by how likely they let a real regression ship.

2. user (2026-06-10T05:59:33.997Z)

```
tests\__init__.py
tests\test_eval_annotations.py
```

tests\test_eval_batch.py
tests\test_eval_classifier.py
tests\test_eval_metrics.py
tests\test_eval_training.py
tests\test_geometry.py
tests\test_providers.py
tests\test_sport_profiles.py
tests\test_video.py
tests\test_annotation_providers.py
tests\test_calibration.py
tests\test_calibrate_wasb.py
tests\test_export_wasb_segments.py
tests\test_gemini_refine.py
tests\test_provider_gemini.py
tests\test_calibrate_local.py
tests\test_distill_local.py
tests\test_automated_provider.py
tests\test_database.py
tests\test_eval_harness.py
tests\test_eval_regression.py
tests\test_gemini.py
tests\test_hit_detector.py
tests\test_rally_gate.py
tests\test_reporting_harvest.py
tests\test_tracknet_hybrid.py
tests\test_validators_base.py
tests\test_wasb_cache.py
tests\test_yolo_hybrid.py
tests\test_base.py
tests\test_config.py
tests\test_hashing.py
tests\test_rally_slicer.py
tests\test_reporting.py
tests\test_reporting_spec_faq.py
tests\test_tracknet_runner.py
tests\test_eval_partial_labels.py
tests\test_rally_seq_proto.py
tests\test_import_smoke.py

3. user (2026-06-10T05:59:34.949Z)

.github\workflows\ci.yml

4. user (2026-06-10T05:59:37.640Z)

backend\eval__init__.py
backend\eval\annotations.py
backend\eval\classifier.py
backend\eval\metrics.py
backend\eval\training.py
backend\providers__init__.py
backend\providers\base.py
backend\sports__init__.py
backend\sports\badminton.py
backend\sports\base.py
backend\utils__init__.py
backend\utils\geometry.py
backend\annotations\base.py
backend\annotations\file_provider.py
backend\annotations\automated_provider.py
backend\annotations__init__.py
backend\tools\rally_slicer.py
backend\tools__init__.py
backend\eval\calibration.py
backend\eval\gemini_refine.py

backend\providers\gemini.py
backend\utils\video.py
backend\eval\audio__init__.py
backend\config__init__.py
backend\eval\audio\e1_probe.py
backend\eval\calibrate_local.py
backend\eval\calibrate_wasb.py
backend\eval\distill_local.py
backend\eval\export_wasb_segments.py
backend\eval\harness.py
backend\eval\regression.py
backend\infrastructure\database.py
backend\pipeline\audio__init__.py
backend\pipeline\audio\hit_detector.py
backend\pipeline\detectors\rally_gate.py
backend\pipeline\segmenters__init__.py
backend\pipeline\segmenters\gemini.py
backend\pipeline\segmenters\yolo_hybrid.py
backend\pipeline\validators\base.py
backend\pipeline\validators__init__.py
backend\pipeline\validators\blur.py
backend\pipeline\validators\format.py
backend\pipeline\validators\pts_continuity.py
backend\pipeline\validators\relevance.py
backend\pipeline\validators\resolution_fps.py
backend\pipeline\validators\scene_cut.py
backend\reporting__init__.py
backend\config\loader.py
backend\config\models.py
backend\eval\batch.py
backend\main.py
backend\pipeline\detectors\base.py
backend\pipeline\detectors\tracknet_runner.py
backend\pipeline\detectors\wasb_infer.py
backend\pipeline\detectors\wasb_runner.py
backend\pipeline\detectors__init__.py
backend\pipeline\segmenters\base.py
backend\pipeline\segmenters\motion.py
backend\pipeline\segmenters\tracknet_hybrid.py
backend\pipeline\segmenters\wasb_hybrid.py
backend\reporting\harvest.py
backend\results.py
backend\tools\export.py
backend\utils\hashing.py
backend\eval\eval_partial_labels.py
backend\eval\rally_seq_proto.py
backend\pipeline\stitcher\compiler.py

5. user (2026-06-10T05:59:37.682Z)

.pytest_cache\README.md
backend\eval__init__.py
backend\eval\annotations.py
backend\eval\classifier.py
backend\eval\metrics.py
backend\eval\training.py
backend\providers__init__.py
backend\providers\base.py
backend\sports__init__.py
backend\sports\badminton.py
backend\sports\base.py
backend\utils__init__.py
backend\utils\geometry.py
docs\EVALUATION.md
docs\TRACKNET_WSL_SETUP.md

pytest.ini
scratch\analyze_motion.py
scratch\clean_state.py
scratch\diagnose_rally.py
scratch\gemini_video_debugger.py
scratch\list_models.py
scratch\test_thresholds.py
tests__init__.py
tests\test_eval_annotations.py
tests\test_eval_batch.py
tests\test_eval_classifier.py
tests\test_eval_metrics.py
tests\test_eval_training.py
tests\test_geometry.py
tests\test_providers.py
tests\test_sport_profiles.py
tests\test_video.py
scripts\generate_khelacharya_flyer.py
backend\annotations\base.py
backend\annotations\file_provider.py
tests\test_annotation_providers.py
backend\annotations\automated_provider.py
backend\annotations__init__.py
docs\GOLDEN_SET_IMPLEMENTATION.md
docs\GOLDEN_SET_PHASE1_VIDEOS.md
backend\tools\rally_slicer.py
backend\tools__init__.py
backend\eval\calibration.py
docs\HOW_RALLY_DETECTION_WORKS.md
tests\test_calibration.py
tests\test_calibrate_wasb.py
backend\eval\gemini_refine.py
backend\providers\gemini.py
backend\utils\video.py
tests\test_export_wasb_segments.py
tests\test_gemini_refine.py
tests\test_provider_gemini.py
tests\test_calibrate_local.py
tests\test_distill_local.py
backend\eval\audio__init__.py
backend\config__init__.py
backend\eval\audio\e1_probe.py
backend\eval\calibrate_local.py
backend\eval\calibrate_wasb.py
backend\eval\distill_local.py
backend\eval\export_wasb_segments.py
backend\eval\harness.py
backend\eval\regression.py
backend\infrastructure\database.py
backend\pipeline\audio__init__.py
backend\pipeline\audio\hit_detector.py
backend\pipeline\detectors\rally_gate.py
backend\pipeline\segmenters__init__.py
backend\pipeline\segmenters\gemini.py
backend\pipeline\segmenters\yolo_hybrid.py
backend\pipeline\validators\base.py
backend\pipeline\validators__init__.py
backend\pipeline\validators\blur.py
backend\pipeline\validators\format.py
backend\pipeline\validators\pts_continuity.py
backend\pipeline\validators\relevance.py
backend\pipeline\validators\resolution_fps.py
backend\pipeline\validators\scene_cut.py
backend\reporting__init__.py
scratch\test_ai_segmenter.py

```

setup.py
tests\test_automated_provider.py
tests\test_database.py
tests\test_eval_harness.py
tests\test_eval_regression.py
tests\test_gemini.py
tests\test_hit_detector.py
tests\test_rally_gate.py
tests\test_reporting_harvest.py
tests\test_tracknet_hybrid.py
tests\test_validators_base.py
tests\test_wasb_cache.py
tests\test_yolo_hybrid.py
run_all_tests.py
backend\config\loader.py
backend\config\models.py
TEST_RUN_SUMMARY.md
backend\eval\batch.py
backend\main.py
backend\pipeline\detectors\base.py
(Results are truncated. Consider using a more specific path or pattern.)

```

6. user (2026-06-10T05:59:42.751Z)

```

1      [pytest]
2      # When `pytest` is invoked without an explicit path, only collect from tests/.
3      # scratch/ holds debugging scripts that hit real APIs (Gemini upload, etc.)
4      # and should never run as part of the regular test suite.
5      testpaths = tests
6

```

7. user (2026-06-10T05:59:43.987Z)

```

1      #!/usr/bin/env python
2      """Run the FULL test suite for the badminton-highlight-indexer.
3
4      A single, discoverable entry point so you (or a reviewer / CI) can verify the
5      whole suite before opening a PR:
6
7          python run_all_tests.py          # full suite, quiet
8          python run_all_tests.py -v      # verbose (passthrough to pytest)
9          python run_all_tests.py -k config # filter by keyword (passthrough)
10         python run_all_tests.py --cov=backend --cov-report=term-missing # coverage
11
12     Anything after the script name is passed straight through to pytest.
13
14     Notes
15     -----
16     - The suite is fully mocked (no GPU, no network, no WSL) and runs offline.
17     - The observability **reporting** tests skip cleanly if the optional ``obsreport``
18       package isn't installed (they use ``pytest.importorskip``). To exercise them too:
19         pip install -e ../sports-obsreport --no-build-isolation
20     - Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all green).
21     """
22     from __future__ import annotations
23
24     import importlib.util
25     import subprocess
26     import sys
27
28
29     def _installed(module: str) -> bool:
30         try:
31             return importlib.util.find_spec(module) is not None
32         except (ImportError, ValueError):

```

```

33         return False
34
35
36     def main() -> int:
37         args = list(sys.argv[1:])
38         # Default to quiet unless the caller chose a verbosity.
39         if not any(a in ("-q", "-v", "-vv", "-vvv", "--quiet", "--verbose") for a in args):
40             args = ["-q", *args]
41         cmd = [sys.executable, "-m", "pytest", *args]
42
43         obs = "available" if _installed("obsreport") else "NOT installed ? reporting tests
will SKIP"
44         bar = "=" * 72
45         print(bar)
46         print(" Full test suite: " + " ".join(cmd))
47         print(f" obsreport (optional reporting dep): {obs}")
48         print(bar, flush=True)
49
50         rc = subprocess.run(cmd).returncode
51
52         print(bar)
53         print(" RESULT: " + ("PASS - all tests green" if rc == 0 else f"FAIL - pytest exit
{rc}"))
54         print(bar)
55         return rc
56
57
58     if __name__ == "__main__":
59         raise SystemExit(main())
60

```

8. user (2026-06-10T05:59:43.988Z)

```

1     # Test Run Diagnosis & Optimizations Summary
2
3     > **Historical note:** This is a point-in-time diagnosis from a May 2026 local test run,
4     > written *before* the layered-directory restructure (PR #40). Path references below
5     > reflect the old flat layout (e.g. a single `segmenter.py`, now the
6     > `backend/pipeline/segmenters/` package). The diagnostic findings and optimization
7     > lessons remain valid; treat the file paths and install commands as historical.
8
9     *Date: May 18, 2026*
10    *Target File: `C:\Users\avidu\Videos\LargeTestVideo.MP4` (HEVC H.265, Size: ~11.5 GB)*
11
12    This summary details the findings, architectural lessons, and performance optimizations
13    implemented during today's local test run of the Badminton Highlight & Rally Indexer.
14    ---
15
16    ## ? Why the Initial Test Run Blocked (and How We Resolved It)
17
18    During the raw test execution on the 11.5 GB video stream, the system hit three
19    distinct, high-impact constraints. We fully diagnosed, optimized, and hardened the pipelines to
20    address them:
21
22    ### 1. The Keyframe Extraction Gap (H.265 Stream Header Parsing)
23    * **The Issue**: The `PTSContinuityValidator` check initially failed with `[FAIL] No
keyframes could be extracted`.
24    * **The Root Cause**: GoPro raw H.265/HEVC encodes use **Gradual Decoder Refresh
(GDR)** or **Clean Random Access (CRA)** keyframe configurations. Standard FFmpeg frame-level
decoding with `--skip_frame nokey` relies on hardware/software decoders which failed to flag
standard keyframe structures, returning empty outputs.
25    * **The Optimization**: We changed the validator to query packet indexes
(`packet=pts_time,flags`) at the **container demuxer level** instead of full frame decoding.
Because packet flags (e.g. `K__`) are written directly to container headers, this bypasses the

```

```

video decoder entirely. It runs **100x faster** and is 100% robust across HEVC containers!
24
25     ### 2. The Blurriness Quality Barrier (Indoor Arena Low-Light Noise)
26     * **The Issue**: The video failed the focus threshold check with a sharpness score of
`35.3` (minimum threshold was `100.0`).
27     * **The Root Cause**: High-speed action footage in typical indoor badminton
environments suffers from high-speed motion blur and ISO sensor noise, reducing the computed
Laplacian variance.
28     * **The Fix**: We lowered the `min_blur_threshold` default inside `config.json` to
`20.0` so that raw fast-moving arena footage passes quality checks cleanly.
29
30     ### 3. OpenCV Seeking Performance Bottleneck
31     * **The Issue**: The segmenter process consumed massive CPU time during early stages,
appearing to respond slowly.
32     * **The Root Cause**: Seeking to arbitrary frames in highly compressed, high-bitrate
large H.265/HEVC files using OpenCV's `cap.set(cv2.CAP_PROP_POS_FRAMES)` is an extremely
expensive seek operation. Because the decoder has to seek to the closest keyframe and decode
forward to the target frame for every sub-sampled frame, it created an  $O(N^2)$  frame-decoding
loop.
33     * **The Optimization**: We refactored the segmenter (then `segmenter.py`, now the
`backend/pipeline/segmenters/` package) to use a high-performance **sequential grab-and-retrieve
pipeline**. By skipping intermediate frames using `cap.grab()` (which parses container
structures without decoding pixel values) and only calling `cap.read()` for our 5 FPS analysis
frames sequentially, we avoided seeks entirely. This delivers a **100x speedup** on multi-
gigabyte MP4s!
34
35     ---
36
37     ## ?? Newly Added Debug Options: `--max-frames`
38
39     To allow you to quickly test video subsets without waiting to process an entire multi-
gigabyte video file, we added the `--max-frames <N>` argument (and `max_frames` in the API
payload):
40     * This restricts the segmenter frame loop to analyze exactly the first  $N$  sub-sampled
frames (at 5 FPS, `--max-frames 300` executes only the first 60 seconds of play).
41     * It immediately exits the loop at the limit and proceeds to segment, cluster, and
seed the database with whatever play timelines it identified in that subset.
42     * This lets you debug whether your visual indices and relevance checks are executing
correctly in **seconds** instead of minutes!
43
44     ---
45
46     ## ? Guidelines for Running on a More Powerful Machine
47
48     Checking this repository into your private GitHub will let you execute the indexer on a
high-powered machine. Here is how to maximize performance:
49
50     1. **CUDA/GPU Hardware Acceleration**:
51         If your other machine has a CUDA-enabled NVIDIA GPU, install PyTorch with CUDA:
52         ```bash
53         pip install torch torchvision --index-url https://download.pytorch.org/whl/cu124
54         ```
55         (`setup.py` auto-selects the matching CUDA wheel ? currently `cu124` ? when it
detects an NVIDIA GPU.)
56         Our diagnostics checker (`setup.py`) will automatically detect CUDA and leverage
standard GPU matrix operations to perform accelerated image difference thresholding!
57     2. **Unbuffered Console Outputs**:
58         To see log statements instantly without OS shell output buffers, use Python's
unbuffered command:
59         ```bash
60         python -u -m backend.main --video-path "C:\Users\avidu\Videos\LargeTestVideo.MP4"
--max-frames 500
61         ```
62

```

9. user (2026-06-10T05:59:44.138Z)

```
1      name: CI
2
3      on:
4        push:
5          branches: [master]
6        pull_request:
7
8      permissions:
9        contents: read
10
11     concurrency:
12       group: ci-${{ github.ref }}
13       cancel-in-progress: true
14
15     jobs:
16       # Fast guard against the 7fbb0 class of breakage: a SyntaxError or
17       # module-level import error shipping while the suite stays green.
18       # Deliberately does NOT install cv2/torch/numpy ? the smoke tests must hold
19       # on machines without the GPU stack (test_import_smoke stubs what's absent).
20       import-smoke:
21         name: Syntax sweep + import smoke (no GPU stack)
22         runs-on: ubuntu-latest
23         steps:
24           - uses: actions/checkout@v4
25           - uses: actions/setup-python@v5
26             with:
27               python-version: "3.13"
28           - name: Install minimal deps
29             run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
30genai
31           - name: Run import smoke tests
32             run: python -m pytest tests/test_import_smoke.py -v
33
34       full-suite:
35         name: Full test suite (offline, mocked)
36         runs-on: ubuntu-latest
37         steps:
38           - uses: actions/checkout@v4
39           - uses: actions/setup-python@v5
40             with:
41               python-version: "3.13"
42               cache: pip
43           - name: System libs for opencv-python
44             run: |
45               sudo apt-get update
46               sudo apt-get install -y libgl1 libglib2.0-0t64 || sudo apt-get install -y
47libgl1 libglib2.0-0
48           - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
49             run: python -m pip install torch torchvision --index-url
50https://download.pytorch.org/whl/cpu
51           - name: Install requirements
52             run: python -m pip install -r requirements.txt pytest
53           # obsreport (private, soft dep) is absent here: reporting tests skip via
54           # pytest.importorskip ? run_all_tests.py prints which.
55           - name: Run full suite
56             run: python run_all_tests.py
```

10. assistant (2026-06-10T05:59:51.018Z)

CI exists (`.github/workflows/ci.yml`, two jobs). Now I'll run the suite in the background while inventorying tests.

11. user (2026-06-10T05:59:52.934Z)

Command running in background with ID: b8pckbll4. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\tasks\b8pckbll4.output. You will be notified when it completes. To check interim output, use Read on that file path.

12. user (2026-06-10T05:59:54.158Z)

```
1 tests/__init__.py
12 tests/test_providers.py
26 tests/test_sport_profiles.py
27 tests/test_gemini_refine.py
27 tests/test_video.py
30 tests/test_calibrate_local.py
35 tests/test_distill_local.py
41 tests/test_rally_gate.py
47 tests/test_geometry.py
48 tests/test_base.py
50 tests/test_hashing.py
52 tests/test_calibrate_wasb.py
55 tests/test_import_smoke.py
60 tests/test_eval_classifier.py
63 tests/test_eval_training.py
67 tests/test_eval_batch.py
67 tests/test_reporting_spec_faq.py
71 tests/test_hit_detector.py
71 tests/test_rally_seq_proto.py
72 tests/test_validators_base.py
75 tests/test_provider_gemini.py
80 tests/test_eval_partial_labels.py
80 tests/test_gemini.py
88 tests/test_annotation_providers.py
91 tests/test_export_wasb_segments.py
93 tests/test_eval_annotations.py
95 tests/test_automated_provider.py
95 tests/test_calibration.py
107 tests/test_reporting.py
109 tests/test_rally_slicer.py
122 tests/test_eval_regression.py
131 tests/test_config.py
143 tests/test_eval_harness.py
143 tests/test_eval_metrics.py
164 tests/test_database.py
171 tests/test_tracknet_hybrid.py
171 tests/test_wasb_cache.py
190 tests/test_reporting_harvest.py
232 tests/test_tracknet_runner.py
320 tests/test_yolo_hybrid.py
3622 total
---
```

tests/__init__.py	0
tests/test_base.py	1
tests/test_providers.py	1
tests/test_calibrate_local.py	2
tests/test_distill_local.py	2
tests/test_gemini.py	2
tests/test_import_smoke.py	2
tests/test_reporting_spec_faq.py	2
tests/test_sport_profiles.py	2
tests/test_calibrate_wasb.py	3
tests/test_rally_seq_proto.py	3
tests/test_validators_base.py	3
tests/test_eval_partial_labels.py	4
tests/test_gemini_refine.py	4

```
tests/test_hashing.py:4
tests/test_provider_gemini.py:4
tests/test_video.py:4
tests/test_geometry.py:5
tests/test_rally_gate.py:5
tests/test_reporting.py:5
tests/test_eval_classifier.py:6
tests/test_export_wasb_segments.py:6
tests/test_config.py:7
tests/test_eval_training.py:7
tests/test_hit_detector.py:7
tests/test_yolo_hybrid.py:7
tests/test_eval_batch.py:8
tests/test_tracknet_hybrid.py:8
tests/test_annotation_providers.py:9
tests/test_automated_provider.py:9
tests/test_calibration.py:9
tests/test_database.py:9
tests/test_eval_regression.py:9
tests/test_eval_annotations.py:11
tests/test_reporting_harvest.py:11
tests/test_wasb_cache.py:11
tests/test_rally_slicer.py:12
tests/test_eval_harness.py:14
tests/test_eval_metrics.py:17
tests/test_tracknet_runner.py:21
```

13. assistant (2026-06-10T06:00:12.324Z)

While the suite runs, let me map tests ? modules via imports.

14. user (2026-06-10T06:00:13.965Z)

```
from backend.annotations import annotation_registry
from backend.annotations import annotation_registry, AnnotationProvider
from backend.annotations.automated_provider import (
from backend.annotations.base import DONE, FAILED
from backend.annotations.file_provider import FileProvider
from backend.config import load_config, save_default_config, AppConfig
from backend.eval import batch
from backend.eval import calibrate_local as cl
from backend.eval import distill_local as dl
from backend.eval import eval_partial_labels as ep
from backend.eval import export_wasb_segments as ex
from backend.eval import gemini_refine as gr
from backend.eval import rally_seq_proto as rp
from backend.eval import regression
from backend.eval import training
from backend.eval.annotations import parse_timestamp, load_annotations, write_scaffold
from backend.eval.audio.e1_probe import cluster_hits
from backend.eval.calibrate_wasb import (
from backend.eval.calibration import (
from backend.eval.classifier import LogisticRegression
from backend.eval.harness import (
from backend.eval.metrics import (
from backend.infrastructure.database import Database
from backend.pipeline.audio import hit_detector as hd
from backend.pipeline.audio.hit_detector import HitEvent
from backend.pipeline.detectors import rally_gate as rg
from backend.pipeline.detectors import wasb_infer as wi
from backend.pipeline.detectors.base import DetectorRunner
from backend.pipeline.detectors.tracknet_runner import (
from backend.pipeline.detectors.tracknet_runner import TrajectoryPoint
from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
from backend.pipeline.segmenters.base import BasePlaySegmenter, SegmenterRegistry
```

```

from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
from backend.pipeline.validators.base import ValidationRegistry, ValidationResult,
VideoValidator
from backend.pipeline.validators.base import ValidationResult # noqa: E402
from backend.providers import provider_registry
from backend.providers.base import AIVideoProvider
from backend.providers.gemini import GeminiProvider
from backend.reporting import emit_indexer_report, harvest # noqa: E402
from backend.reporting import emit_indexer_report, report_enabled
from backend.reporting.harvest import video_meta_from_context # pure helper; no obsreport dep
from backend.results import Success, Failure, Result # test the new contract
from backend.sports import sport_registry
from backend.sports.badminton import BadmintonSportProfile
from backend.sports.base import SportProfile
from backend.tools.export import basic_stats, segments_to_json, segments_to_csv, segments_to_edl
from backend.tools.rally_slicer import (
from backend.tools.rally_slicer import load_rally_labels
from backend.utils.geometry import get_center, get_distance, get_velocity,
get_velocity_normalised, calculate_iou
from backend.utils.hashing import compute_video_id
from backend.utils.video import get_video_duration, extract_video_chunk

```

15. user (2026-06-10T06:00:17.372Z)

```

== tests/__init__.py
== tests/test_annotation_providers.py
from backend.annotations import annotation_registry, AnnotationProvider
from backend.annotations.base import DONE, FAILED
from backend.annotations.file_provider import FileProvider
== tests/test_automated_provider.py
from backend.annotations import annotation_registry
from backend.annotations.base import DONE, FAILED
from backend.annotations.automated_provider import (
== tests/test_base.py
from backend.pipeline.segmenters.base import BasePlaySegmenter, SegmenterRegistry
from backend.results import Success, Failure, Result # test the new contract
== tests/test_calibrate_local.py
from backend.eval import calibrate_local as cl
== tests/test_calibrate_wasb.py
from backend.eval.calibrate_wasb import (
== tests/test_calibration.py
from backend.eval.calibration import (
== tests/test_config.py
from backend.config import load_config, save_default_config, AppConfig
== tests/test_database.py
from backend.infrastructure.database import Database
== tests/test_distill_local.py
from backend.eval import distill_local as dl
from backend.eval.classifier import LogisticRegression
== tests/test_eval_annotations.py
from backend.eval.annotations import parse_timestamp, load_annotations, write_scaffold
== tests/test_eval_batch.py
from backend.eval import batch
== tests/test_eval_classifier.py
from backend.eval.classifier import LogisticRegression
== tests/test_eval_harness.py
from backend.infrastructure.database import Database
from backend.eval.harness import (
== tests/test_eval_metrics.py
from backend.eval.metrics import (
== tests/test_eval_partial_labels.py
from backend.eval import eval_partial_labels as ep
== tests/test_eval_regression.py

```

```

from backend.infrastructure.database import Database
from backend.eval import regression
== tests/test_eval_training.py
from backend.eval import training
== tests/test_export_wasb_segments.py
from backend.eval import export_wasb_segments as ex
from backend.tools.rally_slicer import load_rally_labels
== tests/test_gemini.py
from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
== tests/test_gemini_refine.py
from backend.eval import gemini_refine as gr
== tests/test_geometry.py
from backend.utils.geometry import get_center, get_distance, get_velocity,
get_velocity_normalised, calculate_iou
== tests/test_hashing.py
from backend.utils.hashing import compute_video_id
== tests/test_hit_detector.py
from backend.pipeline.audio import hit_detector as hd
from backend.pipeline.audio.hit_detector import HitEvent
from backend.eval.audio.e1_probe import cluster_hits
== tests/test_import_smoke.py
    "import importlib.util, sys\n"
    "    if importlib.util.find_spec(name) is None:\n"
== tests/test_provider_gemini.py
from backend.providers.gemini import GeminiProvider
== tests/test_providers.py
from backend.providers import provider_registry
from backend.providers.base import AIVideoProvider
from backend.providers.gemini import GeminiProvider
== tests/test_rally_gate.py
from backend.pipeline.detectors.tracknet_runner import TrajectoryPoint
from backend.pipeline.detectors import rally_gate as rg
== tests/test_rally_seq_proto.py
from backend.eval import rally_seq_proto as rp
== tests/test_rally_slicer.py
from backend.tools.rally_slicer import (
from backend.tools.export import basic_stats, segments_to_json, segments_to_csv, segments_to_edl
== tests/test_reporting.py
from backend.reporting import emit_indexer_report, report_enabled
from backend.reporting.harvest import video_meta_from_context # pure helper; no obsreport dep
(see test_reporting_harvest.py which uses pytest.importorskip). Those tests
== tests/test_reporting_harvest.py
pytest.importorskip("obsreport")
from backend.reporting import emit_indexer_report, harvest # noqa: E402
from backend.pipeline.validators.base import ValidationResult # noqa: E402
== tests/test_reporting_spec_faq.py
pytest.importorskip("obsreport")
== tests/test_sport_profiles.py
from backend.sports import sport_registry
from backend.sports.base import SportProfile
from backend.sports.badminton import BadmintonSportProfile
== tests/test_tracknet_hybrid.py
from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
== tests/test_tracknet_runner.py
from backend.pipeline.detectors.tracknet_runner import (
from backend.pipeline.detectors.base import DetectorRunner
from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
== tests/test_validators_base.py
from backend.pipeline.validators.base import ValidationRegistry, ValidationResult,
VideoValidator
== tests/test_video.py
from backend.utils.video import get_video_duration, extract_video_chunk
== tests/test_wasb_cache.py
from backend.pipeline.detectors import wasb_infer as wi

```

```
== tests/test_yolo_hybrid.py
from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
```

16. user (2026-06-10T06:00:26.616Z)

```
1      =====
2      Full test suite: C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe -m
pytest -q
3      obsreport (optional reporting dep): available
4      =====
5      ..... [ 28%]
6      ..... [ 56%]
7      ..... [ 84%]
8      ..... [100%]
9      256 passed in 20.66s
10     =====
11     RESULT: PASS - all tests green
12     =====
13
14     real    0m22.724s
15     user    0m0.030s
16     sys     0m0.092s
17
```

17. assistant (2026-06-10T06:00:46.057Z)

Suite: 256 passed, 0 failed, 0 skipped in 20.66s (obsreport installed locally so reporting tests ran). Now coverage measurement and quality sampling.

18. user (2026-06-10T06:00:54.000Z)

```
pytest-cov ok
numpy 2.4.6
```

19. user (2026-06-10T06:00:54.020Z)

```
1      """Import/syntax smoke tests for the backend package.
2
3      Guards against a SyntaxError (or module-level import error) shipping while the
4      suite stays green: most of the suite imports individual backend modules, so a
5      broken file that nothing imports (e.g. the stitcher compiler, which only
6      backend.main pulls in) used to go unnoticed.
7
8      Both tests are offline and dependency-light: the syntax sweep never imports
9      anything, and the backend.main import runs in a subprocess with any missing
10     heavy deps (cv2/torch/numpy) stubbed, so they also pass in environments
11     without the GPU stack.
12     """
13
14     import py_compile
15     import subprocess
16     import sys
17     from pathlib import Path
18
19     REPO_ROOT = Path(__file__).resolve().parents[1]
20
21
22     def test_backend_package_has_no_syntax_errors():
23         """Every .py file under backend/ must byte-compile."""
24         failures = []
25         for path in sorted((REPO_ROOT / "backend").rglob("*.py")):
26             try:
27                 py_compile.compile(str(path), doraise=True)
28             except py_compile.PyCompileError as exc:
```

```

29         failures.append(f"{path.relative_to(REPO_ROOT)}: {exc.msg}")
30     assert not failures, "Syntax errors in backend/:\n" + "\n".join(failures)
31
32
33 def test_backend_main_imports():
34     """backend.main (FastAPI app, /api/compile) must be importable.
35
36     Runs in a subprocess so stubbing missing heavy deps cannot leak into this
37     pytest process; cwd=repo root puts the package on sys.path.
38     """
39     prelude = (
40         "import importlib.util, sys\n"
41         "from unittest import mock\n"
42         "for name in ('cv2', 'torch', 'numpy'):\n"
43         "    if importlib.util.find_spec(name) is None:\n"
44         "        sys.modules[name] = mock.MagicMock()\n"
45         "import backend.main\n"
46     )
47     result = subprocess.run(
48         [sys.executable, "-c", prelude],
49         cwd=str(REPO_ROOT),
50         capture_output=True,
51         text=True,
52     )
53     assert result.returncode == 0, (
54         "import backend.main failed:\n" + result.stderr.strip()
55     )
56

```

20. user (2026-06-10T06:01:10.053Z)

cv2 4.13.0
torch 2.6.0+cu124

21. user (2026-06-10T06:01:17.123Z)

Command running in background with ID: bwv0cz1wv. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\tasks\bwv0cz1wv.output. You will be notified when it completes. To check interim output, use Read on that file path.

22. user (2026-06-10T06:01:18.068Z)

```

1     import pytest
2     from backend.providers import provider_registry
3     from backend.providers.base import AIVideoProvider
4     from backend.providers.gemini import GeminiProvider
5
6     def test_provider_registry():
7         available = provider_registry.list_available()
8         assert "gemini" in available
9
10        # Retrieve provider instance
11        provider = provider_registry.get("gemini", api_key="dummy",
model_name="gemini-2.5-flash")
12        assert isinstance(provider, GeminiProvider)
13

```

23. user (2026-06-10T06:01:18.427Z)

```

1     import pytest
2     from backend.pipeline.segmenters.base import BasePlaySegmenter, SegmenterRegistry
3     from backend.results import Success, Failure, Result # test the new contract
4

```

```

5     class DummySegmenter(BasePlaySegmenter):
6         @property
7         def name(self) -> str:
8             return "dummy"
9
10        @property
11        def description(self) -> str:
12            return "Dummy segmenter for testing."
13
14        def process_video(self, video_id, video_path, player_pool=None, max_frames=None) ->
Result:
15            # Demonstrate both success and failure paths for the contract
16            if video_path == "fail":
17                return Failure(message="Simulated failure for test", is_recoverable=True)
18            return Success(value=[{"start_time": 0.0, "end_time": 1.0}])
19
20    def test_registry():
21        registry = SegmenterRegistry()
22        registry.register(DummySegmenter)
23
24        available = registry.list_available()
25        assert "dummy" in available
26        assert available["dummy"] == "Dummy segmenter for testing."
27
28        cls = registry.get("dummy")
29        assert cls == DummySegmenter
30
31        inst = cls(None, {})
32        assert inst.name == "dummy"
33        success = inst.process_video("v1", "path")
34        assert isinstance(success, Success)
35        assert success.is_success() is True
36        assert success.value == [{"start_time": 0.0, "end_time": 1.0}]
37        assert success.unwrap() == [{"start_time": 0.0, "end_time": 1.0}]
38
39        failure = inst.process_video("v1", "fail")
40        assert isinstance(failure, Failure)
41        assert failure.is_success() is False
42        assert failure.is_recoverable is True
43        assert "Simulated failure" in failure.message
44        try:
45            failure.unwrap()
46            assert False, "Should have raised"
47        except RuntimeError as e:
48            assert "Simulated failure" in str(e)
49

```

24. user (2026-06-10T06:01:22.868Z)

```

1     import pytest
2     from unittest.mock import MagicMock, patch
3     from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
4
5
6     def _make_cap_mock(num_frames=300):
7         """A cv2.VideoCapture mock yielding `num_frames` readable frames at 30fps."""
8         import cv2
9         cap_mock = MagicMock()
10        cap_mock.isOpened.return_value = True
11
12        def mock_get_prop(prop):
13            if prop == cv2.CAP_PROP_FPS:
14                return 30.0
15            if prop == cv2.CAP_PROP_FRAME_WIDTH:
16                return 1000.0

```

```

17         return 0.0
18
19     cap_mock.get.side_effect = mock_get_prop
20     cap_mock.read.side_effect = [(True, "frame")] * num_frames + [(False, None)]
21     return cap_mock
22
23
24     def _make_detections(num_frames=300, step=5, track_id=1):
25         """Per-frame return values for a mocked _detect_and_track: one steadily-moving
26         person. Each entry is the list of (track_id, bbox) for that frame, where the box
27         advances `step` px/frame so its normalised velocity clears the test thresholds.
28         """
29         out = []
30         for i in range(num_frames):
31             x = i * step
32             out.append([(track_id, (float(x), 0.0, float(x + 10), 10.0))])
33         return out
34
35
36     def _mock_stage1(segmenter, num_frames=300, step=5):
37         """Wire a segmenter's Stage-1 detection+tracking to deterministic fakes so tests
38         never need torch model inference, GPU, or a real supervision tracker."""
39         segmenter.model = MagicMock() # short-circuits _lazy_load_model
40         segmenter._make_tracker = MagicMock(return_value=MagicMock())
41         segmenter._detect_and_track = MagicMock(side_effect=_make_detections(num_frames,
step))
42
43
44     @patch("backend.pipeline.segmenters.yolo_hybrid.cv2.VideoCapture")
45     @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
46     @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
47     @patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
48     @patch("os.environ.get")
49     def test_yolo_hybrid_segmenter(mock_env_get, mock_provider_registry, mock_extract,
50                                   mock_get_dur, mock_cap):
51         mock_env_get.return_value = "dummy_api_key"
52         mock_get_dur.return_value = 10.0
53         mock_extract.return_value = True
54
55         # Mock provider returned play segment
56         mock_provider = MagicMock()
57         mock_provider.analyze_video.return_value = ([
58             {"start_time": 0.0, "end_time": 3.0, "shuttle_exchange_count": 2}
59         ], {})
60         mock_provider_registry.get.return_value = mock_provider
61
62         db_mock = MagicMock()
63         db_mock.add_play_segment.return_value = "yolo_segment_1"
64
65         config = {
66             "indexing": {
67                 "use_gpu": False,
68                 "chunk_duration": 100,
69                 "chunk_overlap": 10,
70                 "yolo_velocity_threshold": 0.1,
71                 "dead_time_merge_gap": 2.0,
72                 # Process every frame so all 300 frames are tracked
73                 "yolo_frame_skip": 1,
74                 "min_action_window_duration": 1.0,
75             }
76         }
77
78         segmenter = HybridYoloSegmenter(db_mock, config)
79         _mock_stage1(segmenter)
80         mock_cap.return_value = _make_cap_mock()

```



```

81
82     res = segmenter.process_video("v1", "dummy.mp4")
83
84     assert len(res) == 1
85     assert res[0]["id"] == "yolo_segment_1"
86     assert res[0]["start_time"] == 0.0
87     assert res[0]["end_time"] == 3.0
88
89
90     @patch("backend.pipeline.segmenters.yolo_hybrid.cv2.VideoCapture")
91     @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
92     @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
93     @patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
94     @patch("os.environ.get")
95     def test_yolo_hybrid_no_api_key(mock_env_get, mock_provider_registry, mock_extract,
96                                   mock_get_dur, mock_cap):
97         """Segmenter should return empty when GEMINI_API_KEY is missing."""
98         mock_env_get.return_value = None
99
100         db_mock = MagicMock()
101         config = {"indexing": {"use_gpu": False}}
102
103         segmenter = HybridYoloSegmenter(db_mock, config)
104         segmenter.model = MagicMock()
105
106         res = segmenter.process_video("v1", "dummy.mp4")
107         assert res == []
108
109
110     @patch("backend.pipeline.segmenters.yolo_hybrid.cv2.VideoCapture")
111     @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
112     @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
113     @patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
114     @patch("os.environ.get")
115     def test_yolo_hybrid_drops_bad_timestamps(mock_env_get, mock_provider_registry,
116 mock_extract,
117                                               mock_get_dur, mock_cap):
118         """Play segments with unparseable timestamps should be dropped, not silently
119 kept."""
120         mock_env_get.return_value = "dummy_api_key"
121         mock_get_dur.return_value = 10.0
122         mock_extract.return_value = True
123
124         # Provider returns one good segment and one with garbage timestamps
125         mock_provider = MagicMock()
126         mock_provider.analyze_video.return_value = ([
127             {"start_time": 1.0, "end_time": 3.0},
128             {"start_time": "N/A", "end_time": "??"},
129         ], {})
130         mock_provider_registry.get.return_value = mock_provider
131
132         db_mock = MagicMock()
133         db_mock.add_play_segment.return_value = "segment_ok"
134
135         config = {
136             "indexing": {
137                 "use_gpu": False,
138                 "chunk_duration": 100,
139                 "chunk_overlap": 10,
140                 "yolo_velocity_threshold": 0.05,
141                 "dead_time_merge_gap": 2.0,
142                 "yolo_frame_skip": 1,
143                 "min_action_window_duration": 0.5,
144             }
145         }

```

```

144
145     segmenter = HybridYoloSegmenter(db_mock, config)
146     _mock_stage1(segmenter)
147     mock_cap.return_value = _make_cap_mock()
148
149     res = segmenter.process_video("v1", "dummy.mp4")
150
151     # Only the valid segment should survive ? the "N/A" one should be dropped
152     assert len(res) == 1
153     assert res[0]["start_time"] == 1.0
154
155
156 @patch("backend.pipeline.segmenters.yolo_hybrid.cv2.VideoCapture")
157 def test_extract_windows_returns_enriched_dicts(mock_cap):
158     """_extract_action_windows_from_chunk should return dicts with motion stats."""
159     config = {"indexing": {"min_action_window_duration": 1.0}}
160     segmenter = HybridYoloSegmenter(MagicMock(), config)
161     _mock_stage1(segmenter)
162     mock_cap.return_value = _make_cap_mock()
163
164     windows = segmenter._extract_action_windows_from_chunk(
165         "chunk.mp4", chunk_start=5.0, velocity_thresh=0.05, merge_gap=2.0,
166         frame_skip=1, chunk_index=2,
167     )
168
169     assert len(windows) >= 1
170     w = windows[0]
171     assert set(w) >= {"start", "end", "active_frame_count", "peak_velocity",
172                     "mean_velocity", "track_id_count", "chunk_index"}
173     assert w["chunk_index"] == 2
174     assert w["start"] >= 5.0 # offset into full-video coordinates
175     assert w["active_frame_count"] > 0
176     assert w["peak_velocity"] >= w["mean_velocity"] > 0
177     assert w["track_id_count"] == 1
178
179
180 @patch("backend.pipeline.segmenters.yolo_hybrid.cv2.VideoCapture")
181 @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
182 @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
183 @patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
184 @patch("os.environ.get")
185 def test_yolo_hybrid_persists_and_links_candidates(mock_env_get, mock_provider_registry,
186                                                    mock_extract, mock_get_dur,
187 mock_cap):
188     """Raw candidates should be stored, and confirmed ones linked to their segment."""
189     mock_env_get.return_value = "dummy_api_key"
190     mock_get_dur.return_value = 10.0
191     mock_extract.return_value = True
192
193     mock_provider = MagicMock()
194     mock_provider.analyze_video.return_value = ([
195         {"start_time": 0.0, "end_time": 3.0, "shuttle_exchange_count": 2}
196     ], {})
197     mock_provider_registry.get.return_value = mock_provider
198
199     db_mock = MagicMock()
200     db_mock.add_candidate_segment.return_value = 77
201     db_mock.add_play_segment.return_value = 101
202
203     config = {
204         "indexing": {
205             "use_gpu": False,
206             "chunk_duration": 100,
207             "chunk_overlap": 10,
208             "yolo_velocity_threshold": 0.05,

```

```

208         "dead_time_merge_gap": 2.0,
209         "yolo_frame_skip": 1,
210         "min_action_window_duration": 1.0,
211     }
212 }
213
214 segmenter = HybridYoloSegmenter(db_mock, config)
215 _mock_stage1(segmenter)
216 mock_cap.return_value = _make_cap_mock()
217
218 res = segmenter.process_video("v1", "dummy.mp4")
219
220 assert len(res) == 1
221 # Candidate persisted with the right detector + reproducibility params
222 assert db_mock.add_candidate_segment.called
223 _, kwargs = db_mock.add_candidate_segment.call_args
224 assert kwargs["detector"] == "yolo_hybrid"
225 assert "peak_velocity" in kwargs["stats"]
226 assert kwargs["detection_params"]["frame_skip"] == 1
227 # Confirmed segment linked back to its candidate
228 db_mock.link_candidate_to_segment.assert_called_once_with(77, 101)
229
230
231 @patch("backend.pipeline.segmenters.yolo_hybrid.cv2.VideoCapture")
232 @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
233 @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
234 @patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
235 @patch("os.environ.get")
236 def test_yolo_hybrid_store_candidates_disabled(mock_env_get, mock_provider_registry,
237                                               mock_extract, mock_get_dur, mock_cap):
238     """store_yolo_candidates=False should skip DB persistence (and linking)."""
239     mock_env_get.return_value = "dummy_api_key"
240     mock_get_dur.return_value = 10.0
241     mock_extract.return_value = True
242
243     mock_provider = MagicMock()
244     mock_provider.analyze_video.return_value = ([
245         {"start_time": 0.0, "end_time": 3.0}
246     ], {})
247     mock_provider_registry.get.return_value = mock_provider
248
249     db_mock = MagicMock()
250     db_mock.add_play_segment.return_value = 101
251
252     config = {
253         "indexing": {
254             "use_gpu": False,
255             "chunk_duration": 100,
256             "chunk_overlap": 10,
257             "yolo_velocity_threshold": 0.05,
258             "dead_time_merge_gap": 2.0,
259             "yolo_frame_skip": 1,
260             "min_action_window_duration": 1.0,
261             "store_yolo_candidates": False,
262         }
263     }
264
265     segmenter = HybridYoloSegmenter(db_mock, config)
266     _mock_stage1(segmenter)
267     mock_cap.return_value = _make_cap_mock()
268
269     segmenter.process_video("v1", "dummy.mp4")
270
271     db_mock.add_candidate_segment.assert_not_called()
272     db_mock.link_candidate_to_segment.assert_not_called()

```

```

273
274
275 def test_detect_and_track_filters_persons_and_returns_track_ids():
276     """_detect_and_track should keep only confident PERSON detections, feed them to
277     the tracker, and return (track_id, bbox) tuples ? exercising the real torchvision
278     output-shape handling and the COCO person==1 filter (the YOLO-was-0 gotcha)."""
279     import numpy as np
280     from backend.pipeline.segmenters.yolo_hybrid import _PERSON_CLASS_ID
281
282     segmenter = HybridYoloSegmenter(MagicMock(), {"indexing": {}})
283     segmenter.device = "cpu"
284
285     # Fake torchvision detector output: a person (kept), a low-score person (dropped),
286     # and a racket/other class (dropped).
287     out = {
288         "boxes": MagicMock(),
289         "labels": MagicMock(),
290         "scores": MagicMock(),
291     }
292     out["boxes"].detach.return_value.cpu.return_value.numpy.return_value = np.array(
293         [[0, 0, 10, 10], [5, 5, 15, 15], [20, 20, 30, 30]], dtype=float)
294     out["labels"].detach.return_value.cpu.return_value.numpy.return_value = np.array(
295         [_PERSON_CLASS_ID, _PERSON_CLASS_ID, 38]) # 38 = sports-ball-ish, not person
296     out["scores"].detach.return_value.cpu.return_value.numpy.return_value = np.array(
297         [0.9, 0.1, 0.99])
298     segmenter.model = MagicMock(return_value=out)
299
300     # Fake tracker echoes the detections it receives, assigning a single track id.
301     tracker = MagicMock()
302     tracked = MagicMock()
303     tracked.__len__.return_value = 1
304     tracked.tracker_id = [7]
305     tracked.xyxy = np.array([[0, 0, 10, 10]], dtype=float)
306     tracker.update_with_detections.return_value = tracked
307
308     with patch("backend.pipeline.segmenters.yolo_hybrid.cv2.cvtColor",
309 return_value=np.zeros((10, 10, 3))), \
310         patch("backend.pipeline.segmenters.yolo_hybrid.torch") as mock_torch, \
311         patch.dict("sys.modules", {"supervision": MagicMock()}):
312         mock_torch.no_grad.return_value.__enter__ = lambda s: None
313         mock_torch.no_grad.return_value.__exit__ = lambda s, *a: None
314         # torch.from_numpy(...).permute(...).float().div(...).to(...) -> dummy tensor
315         mock_torch.from_numpy.return_value.permute.return_value.float.return_value.div.r
316         eturn_value.to.return_value = "tensor"
317         result = segmenter._detect_and_track("frame", tracker, score_thresh=0.5)
318
319     # Only the high-confidence person survived the score+class filter
320     assert result == [(7, (0.0, 0.0, 10.0, 10.0))]
321     # And exactly that filtered set (1 box) was handed to the tracker
322     tracker.update_with_detections.assert_called_once()
323

```

25. user (2026-06-10T06:01:23.419Z)

```

1     """Tests for the TrackNetHybridSegmenter pipeline. The WSL/TensorFlow runner is
2     mocked out (TrackNetRunner), so these run anywhere ? they exercise Phase 1-4 glue:
3     healthcheck gating, candidate persistence, AI handoff, and DB population/linking."""
4
5     import cv2
6     import pytest
7     from unittest.mock import MagicMock, patch
8
9     from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
10
11

```

```

12     def _window(start=1.0, end=4.0):
13         return {
14             "start": start, "end": end, "active_frame_count": 30,
15             "peak_velocity": 0.4, "mean_velocity": 0.2, "track_id_count": 1,
16             "chunk_index": 0,
17         }
18
19
20     def _patched(fn):
21         """Stack the patches every pipeline test needs. fps/width probing is stubbed
22         so cv2 internals don't need mocking (covered separately).
23
24         Patches are applied innermost-first, and mock.patch injects the innermost
25         patch as the *first* positional arg ? so this order matches the signature
26         (mock_runner_cls, mock_probe, mock_dur, mock_extract, mock_provider_registry,
27         mock_env)."""
28         fn = patch("backend.pipeline.segmenters.tracknet_hybrid.TrackNetRunner")(fn)
29         fn = patch.object(TrackNetHybridSegmenter, "_probe_fps_width", return_value=(30.0,
30         1280.0))(fn)
31         fn = patch("backend.pipeline.segmenters.tracknet_hybrid.get_video_duration")(fn)
32         fn = patch("backend.pipeline.segmenters.tracknet_hybrid.extract_video_chunk")(fn)
33         fn = patch("backend.pipeline.segmenters.tracknet_hybrid.provider_registry")(fn)
34         fn = patch("os.environ.get")(fn)
35         return fn
36
37     def _make_runner(windows=None, csv="out.csv", healthy=True):
38         runner = MagicMock()
39         runner.healthcheck.return_value = (healthy, "TF 2.17 GPUs 1" if healthy else "no
GPU")
40         runner.run_predict.return_value = csv
41         runner.parse_trajectory_csv.return_value = [MagicMock(visible=True)]
42         runner.trajectory_to_action_windows.return_value = windows if windows is not None
43     else [_window()]
44         return runner
45
46     @patched
47     def test_pipeline_happy_path(mock_runner_cls, mock_probe, mock_dur, mock_extract,
48                                 mock_provider_registry, mock_env):
49         mock_env.return_value = "dummy_key"
50         mock_dur.return_value = 30.0
51         mock_extract.return_value = True
52         mock_runner_cls.return_value = _make_runner()
53
54         provider = MagicMock()
55         provider.analyze_video.return_value = (
56             [{"start_time": 0.5, "end_time": 3.0, "shuttle_exchange_count": 4}], {})
57         mock_provider_registry.get.return_value = provider
58
59         db = MagicMock()
60         db.add_candidate_segment.return_value = 55
61         db.add_play_segment.return_value = 200
62
63         seg = TrackNetHybridSegmenter(db, {"indexing": {}})
64         res = seg.process_video("v1", "clip.mp4")
65
66         assert len(res) == 1
67         assert res[0]["id"] == 200
68         # Candidate persisted under the tracknet detector name...
69         _, kwargs = db.add_candidate_segment.call_args
70         assert kwargs["detector"] == "tracknet_hybrid"
71         # ...and the confirmed segment linked back to it.
72         db.link_candidate_to_segment.assert_called_once_with(55, 200)
73

```

```

74
75     @patched
76     def test_pipeline_no_api_key_returns_empty(mock_runner_cls, mock_probe, mock_dur,
mock_extract,
77                                         mock_provider_registry, mock_env):
78         mock_env.return_value = None
79         mock_dur.return_value = 30.0
80         mock_runner_cls.return_value = _make_runner()
81
82         seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
83         assert seg.process_video("v1", "clip.mp4") == []
84
85
86     @patched
87     def test_pipeline_healthcheck_failure_aborts(mock_runner_cls, mock_probe, mock_dur,
mock_extract,
88                                         mock_provider_registry, mock_env):
89         mock_env.return_value = "dummy_key"
90         mock_dur.return_value = 30.0
91         mock_extract.return_value = True
92         mock_runner_cls.return_value = _make_runner(healthy=False)
93         mock_provider_registry.get.return_value = MagicMock()
94
95         seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
96         res = seg.process_video("v1", "clip.mp4")
97         assert res == []
98         # Should bail before ever running inference.
99         mock_runner_cls.return_value.run_predict.assert_not_called()
100
101
102     @patched
103     def test_pipeline_no_trajectory_aborts(mock_runner_cls, mock_probe, mock_dur,
mock_extract,
104                                         mock_provider_registry, mock_env):
105         mock_env.return_value = "dummy_key"
106         mock_dur.return_value = 30.0
107         mock_runner_cls.return_value = _make_runner(csv=None) # predict produced nothing
108         mock_provider_registry.get.return_value = MagicMock()
109
110         seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
111         assert seg.process_video("v1", "clip.mp4") == []
112
113
114     @patched
115     def test_pipeline_store_candidates_disabled(mock_runner_cls, mock_probe, mock_dur,
mock_extract,
116                                         mock_provider_registry, mock_env):
117         mock_env.return_value = "dummy_key"
118         mock_dur.return_value = 30.0
119         mock_extract.return_value = True
120         mock_runner_cls.return_value = _make_runner()
121
122         provider = MagicMock()
123         provider.analyze_video.return_value = ([{"start_time": 0.5, "end_time": 3.0}], {})
124         mock_provider_registry.get.return_value = provider
125
126         db = MagicMock()
127         db.add_play_segment.return_value = 200
128
129         seg = TrackNetHybridSegmenter(db, {"indexing": {"store_yolo_candidates": False}})
130         seg.process_video("v1", "clip.mp4")
131
132         db.add_candidate_segment.assert_not_called()
133         db.link_candidate_to_segment.assert_not_called()
134

```

```

135
136     @_patched
137     def test_pipeline_drops_unparseable_segments(mock_runner_cls, mock_probe, mock_dur,
mock_extract,
138                                             mock_provider_registry, mock_env):
139         mock_env.return_value = "dummy_key"
140         mock_dur.return_value = 30.0
141         mock_extract.return_value = True
142         mock_runner_cls.return_value = _make_runner()
143
144         provider = MagicMock()
145         provider.analyze_video.return_value = (
146             [{"start_time": 0.5, "end_time": 3.0}, {"start_time": "N/A", "end_time": "??"}],
147             {})
148         mock_provider_registry.get.return_value = provider
149
150         db = MagicMock()
151         db.add_candidate_segment.return_value = 55
152         db.add_play_segment.return_value = 200
153
154         seg = TrackNetHybridSegmenter(db, {"indexing": {}})
155         res = seg.process_video("v1", "clip.mp4")
156         assert len(res) == 1 # the "N/A" segment is dropped
157
158     def test_probe_fps_width_fallbacks():
159         with patch("backend.pipeline.segmenters.tracknet_hybrid.cv2.VideoCapture") as
mock_cap:
160             bad = MagicMock()
161             bad.isOpened.return_value = False
162             bad.get.return_value = 0.0
163             mock_cap.return_value = bad
164             fps, width = TrackNetHybridSegmenter._probe_fps_width("missing.mp4")
165             assert fps == 30.0 and width == 1280.0
166
167
168     def test_segmenter_identity():
169         seg = TrackNetHybridSegmenter(MagicMock(), {})
170         assert seg.name == "tracknet_hybrid"
171         assert "TrackNet" in seg.description
172

```

26. user (2026-06-10T06:01:34.227Z)

```

1     """Resumable-cache layer of wasb_infer (pure-Python parts; no WSL/GPU).
2
3     Covers the reboot-resume guarantees: per-window incremental cache, cross-window
4     detection accumulation on replay, crash-truncation recovery, manifest
5     compatibility/invalidation, and atomic trajectory writes.
6     """
7     import json
8     import os
9     import os.path as osp
10
11     import numpy as np
12
13     from backend.pipeline.detectors import wasb_infer as wi
14
15
16     def _win(idx, frames):
17         """A window record: frames = [(fid, [[x,y,score,scale], ...]), ...]."""
18         return {"w": idx, "f": [[fid, dets] for fid, dets in frames]}
19
20
21     def _write_windows(cache_dir, windows):

```

```

22     det_jsonl, _ = wi._cache_paths(cache_dir)
23     with open(det_jsonl, "a") as f:
24         wi._append_windows(f, windows)
25
26
27 # ----- #
28 # detection (de)serialization
29 # ----- #
30 def test_det_plain_and_back_roundtrip():
31     det = {"xy": np.array([439.0, 64.5]), "score": np.float32(0.87), "scale": 0}
32     plain = wi._det_plain(det)
33     assert plain == [439.0, 64.5, float(np.float32(0.87)), 0]
34
35     back = wi._dets_to_tracker([plain])
36     assert len(back) == 1
37     assert isinstance(back[0]["xy"], np.ndarray) # tracker does vector math on xy
38     assert back[0]["xy"].tolist() == [439.0, 64.5]
39     assert back[0]["scale"] == 0
40
41
42 # ----- #
43 # cache replay + resume index
44 # ----- #
45 def test_replay_accumulates_detections_across_windows(tmp_path):
46     cache = str(tmp_path / "c")
47     os.makedirs(cache)
48     # frame 1 appears in BOTH windows -> its candidates must accumulate in window order;
49     # frame 2 is empty in window 0 but detected in window 1.
50     _write_windows(cache, [
51         _win(0, [(0, [[10, 20, 0.9, 0]]), (1, [[11, 21, 0.8, 0]]), (2, [])]),
52         _win(1, [(1, [[12, 22, 0.7, 0]]), (2, [[13, 23, 0.6, 0]]), (3, [])]),
53     ])
54     windows_done, det = wi.load_and_clean_cache(cache)
55     assert windows_done == 2
56     assert det[0] == [[10, 20, 0.9, 0]]
57     assert det[1] == [[11, 21, 0.8, 0], [12, 22, 0.7, 0]] # window-0 then window-1
58     assert det[2] == [[13, 23, 0.6, 0]]
59     assert det[3] == [] # seen but never detected
60
61
62 def test_resume_index_equals_window_count(tmp_path):
63     cache = str(tmp_path / "c")
64     os.makedirs(cache)
65     _write_windows(cache, [_win(i, [(i, [[i, i, 0.5, 0]])]) for i in range(5)])
66     windows_done, _ = wi.load_and_clean_cache(cache)
67     assert windows_done == 5
68
69
70 def test_empty_cache(tmp_path):
71     cache = str(tmp_path / "c")
72     os.makedirs(cache)
73     windows_done, det = wi.load_and_clean_cache(cache)
74     assert windows_done == 0 and dict(det) == {}
75
76
77 # ----- #
78 # crash robustness
79 # ----- #
80 def test_truncated_trailing_line_recovered_and_cleaned(tmp_path):
81     cache = str(tmp_path / "c")
82     os.makedirs(cache)
83     _write_windows(cache, [_win(0, [(0, [[1, 1, 0.9, 0]])]),
84                             _win(1, [(1, [[2, 2, 0.8, 0]])])])
85     det_jsonl, _ = wi._cache_paths(cache)
86     with open(det_jsonl, "a") as f: # simulate a crash mid-flush

```



```

87         f.write({'w':2,"f":[[2,[[3,3,']
88     windows_done, det = wi.load_and_clean_cache(cache)
89     assert windows_done == 2                                # only the two clean windows
survive
90     assert det[1] == [[2, 2, 0.8, 0]]
91     # file is rewritten to the clean prefix -> appending window 2 stays contiguous
92     with open(det_jsonl) as f:
93         lines = [json.loads(l) for l in f if l.strip()]
94     assert [o["w"] for o in lines] == [0, 1]
95     _write_windows(cache, [_win(2, [(2, [[3, 3, 0.7, 0]]))])
96     assert wi.load_and_clean_cache(cache)[0] == 3
97
98
99     def test_noncontiguous_window_index_stops_replay(tmp_path):
100         cache = str(tmp_path / "c")
101         os.makedirs(cache)
102         _write_windows(cache, [_win(0, [(0, [[1, 1, 0.9, 0]]))],
103                               [_win(5, [(5, [[2, 2, 0.8, 0]]))]) # gap -> stop after w=0
104         windows_done, det = wi.load_and_clean_cache(cache)
105         assert windows_done == 1
106         assert 5 not in det
107
108
109     # ----- #
110     # manifest: atomic write + compatibility / invalidation
111     # ----- #
112     def test_manifest_roundtrip_and_compat(tmp_path):
113         cache = str(tmp_path / "c")
114         os.makedirs(cache)
115         m = {
116             "version": wi.CACHE_VERSION,
117             "frames_dir": osp.abspath("/x/frames"),
118             "weights": "w.pth.tar",
119             "sport": "badminton",
120             "frames_in": 3,
121             "frames_total": 100,
122             "windows_total": 98,
123             "windows_done": 42,
124         }
125         wi.write_manifest(cache, m)
126         assert not osp.exists(osp.join(cache, wi._MANIFEST_FILE + ".tmp")) # atomic, no
tmp left
127         assert wi.read_manifest(cache) == m
128
129         base = dict(frames_dir="/x/frames", weights="w.pth.tar", sport="badminton",
130                     frames_in=3, frames_total=100)
131         assert wi.manifest_compatible(m, **base)
132         assert not wi.manifest_compatible(m, **{**base, "frames_total": 101}) # more
frames
133         assert not wi.manifest_compatible(m, **{**base, "weights": "other.pth"})
134         assert not wi.manifest_compatible(m, **{**base, "sport": "tennis"})
135         assert not wi.manifest_compatible({}, **base) # no cache
136         assert not wi.manifest_compatible(**m, "version": 999, **base) # schema bump
137
138
139     def test_read_manifest_missing_returns_none(tmp_path):
140         assert wi.read_manifest(str(tmp_path)) is None
141
142
143     def test_reset_cache_removes_files(tmp_path):
144         cache = str(tmp_path / "c")
145         os.makedirs(cache)
146         _write_windows(cache, [_win(0, [(0, [[1, 1, 0.9, 0]]))])
147         wi.write_manifest(cache, {"version": wi.CACHE_VERSION})
148         wi.reset_cache(cache)

```

```

149         det_jsonl, mpath = wi._cache_paths(cache)
150         assert not osp.exists(det_jsonl) and not osp.exists(mpath)
151
152
153     # ----- #
154     # misc helpers
155     # ----- #
156     def test_default_cache_dir_is_next_to_output():
157         out = osp.join("some", "dir", "sample_long_traj.csv")
158         cd = wi.default_cache_dir(out)
159         assert osp.basename(cd) == "sample_long_traj_wasbcache"
160         assert osp.dirname(cd) == osp.dirname(osp.abspath(out))
161
162
163     def test_atomic_write_trajectory(tmp_path):
164         out = str(tmp_path / "out" / "traj.csv")
165         wi._atomic_write_trajectory(out, [(0, 1, 439.0, 64.0), (1, 0, -1, -1)])
166         assert not osp.exists(out + ".tmp")
167         with open(out) as f:
168             text = f.read().splitlines()
169         assert text[0] == "Frame,Visibility,X,Y"
170         assert text[1] == "0,1,439.0,64.0"
171         assert text[2] == "1,0,-1,-1"
172

```

27. user (2026-06-10T06:01:36.244Z)

```

tests/test_reporting_harvest.py:46:         config={"ai_provider": "gemini", "ai_model": "gemini-
flash", "indexing": {"default_segmenter": "wasb_hybrid"}},
tests/test_reporting_harvest.py:51:         segmenter_requested="wasb_hybrid",
tests/test_reporting_harvest.py:52:         segmenter_actual="wasb_hybrid",
tests/test_reporting_harvest.py:107:    cfg = {"ai_provider": "gemini", "indexing":
{"default_segmenter": "wasb_hybrid"}}
tests/test_reporting_harvest.py:150:         config={"ai_provider": "gemini", "indexing":
{"default_segmenter": "wasb_hybrid"}},
tests/test_reporting_harvest.py:152:         segmenter_requested="wasb_hybrid",
segmenter_actual="wasb_hybrid",
tests/test_reporting_harvest.py:174:         segmenter_requested="wasb_hybrid",
segmenter_actual="wasb_hybrid",
Binary file tests/__pycache__/test_reporting_harvest.cpython-313-pytest-9.0.3.pyc matches
Binary file tests/__pycache__/test_reporting_harvest.cpython-313.pyc matches
---
backend/providers/base.py:10:     def analyze_video(self, video_path: str, prompt: str,
backend/providers/gemini.py:28:     def analyze_video(self, video_path: str, prompt: str,

```

28. user (2026-06-10T06:01:44.878Z)

```

1     from abc import ABC, abstractmethod
2     from typing import Tuple
3
4     class AIVideoProvider(ABC):
5         def __init__(self, api_key: str, model_name: str):
6             self.api_key = api_key
7             self.model_name = model_name
8
9         @abstractmethod
10        def analyze_video(self, video_path: str, prompt: str,
11                        chunk_duration: float = 0.0,
12                        label: str = "") -> Tuple[list, dict]:
13            """Sends a video file to the AI model along with a prompt, and returns a tuple
of
14            (segments_list, metrics_dict).
15
16        Args:
17            video_path: Path to the video file to analyze.

```

```

18         prompt: Prompt instructions.
19         chunk_duration: Duration of this chunk in seconds, or 0.0 if not chunked.
20         label: Diagnostic label for logging.
21
22     Returns:
23         Tuple containing:
24             - A list of detected segment dicts (e.g. [{"start_time": ..., "end_time":
...}])
25             - A dictionary of performance metrics (e.g. {"upload_time": ...,
"process_time": ..., "gen_time": ...})
26         """
27         pass
28

```

29. user (2026-06-10T06:01:48.730Z)

```

225 backend/pipeline/segmenters/wasb_hybrid.py
254 backend/pipeline/segmenters/tracknet_hybrid.py
229 backend/pipeline/segmenters/motion.py
426 backend/main.py
221 backend/pipeline/stitcher/compiler.py
150 backend/pipeline/detectors/wasb_runner.py
410 backend/pipeline/detectors/wasb_infer.py
60 backend/results.py
123 backend/tools/export.py
2098 total
---
3,12c3,11
< Same three-phase shape as HybridYoloSegmenter, but Phase 2 (candidate
< generation) uses a XXV4 *shuttle* trajectory instead of YOLO *person*
< motion. XX runs in WSL2 (see backend/detectors/XX_runner.py and
< docs/DECISIONS.md ADR-001); the shuttle trajectory is converted into candidate
< rally windows, which are then handed to the AI provider for semantic boundary
< confirmation (Phase 3) and written to the DB (Phase 4).
<
< Rationale: a tiny, fast, motion-blurred shuttle is exactly what general
< detectors miss; a purpose-built trajectory model localises it far more
< precisely, giving cleaner candidate rallies than person-velocity heuristics.
---
> Identical three-phase shape to XXHybridSegmenter, but Phase 2 uses **XX**
> (nttcom/XX-SBDT, MIT) shuttle-trajectory detection via WSL2
> (backend/detectors/XX_runner.py) instead of XXV4. XX ships pretrained
> badminton weights and runs today, so this is the live shuttle-motion substrate.
> Phase 3 (AI semantic boundaries) and Phase 4 (DB) are shared with the other hybrids.
>
> For the per-video *calibration* goal we mainly use Phase 2 (trajectory -> candidate
> windows); the windowing thresholds (velocity_thresh / merge_gap / min_window_duration)
> are what get tuned against golden rally labels.
14d12
<
47,48c45,46
<         return ("Two-stage pipeline: XXV4 shuttle-trajectory detection (WSL2) for precise "
<             "candidate rallies + AI provider for high-precision semantic boundaries.")
---
>         return ("Two-stage pipeline: XX shuttle-trajectory detection (WSL2, pretrained
badminton "
>             "weights) for precise candidate rallies + AI provider for semantic
boundaries.")
52d49
<         """Return (fps, frame_width) for a video, with sensible fallbacks."""
61d57
<         # --- Sport profile + AI provider wiring (identical to the other segmenters) ---
76,80c72
<         logger.error(
<             f"{api_key_env_var} environment variable is not set! "

```

```

<         f"To run the {provider_name} handoff, set your API key. "
<         f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
<     )
---
>         logger.error(f"{api_key_env_var} not set; cannot run the {provider_name}
handoff.")
94,96c86,87
<     # --- XX runner config + thresholds ---
<     tn_cfg = XXConfig.from_indexing_cfg(idx_cfg)
<     runner: DetectorRunner = XXRunner(tn_cfg)
---
>     XX_cfg = XXConfig.from_indexing_cfg(idx_cfg)
>     runner: DetectorRunner = XXRunner(XX_cfg)
98,99c89,90
<     tn = idx_cfg.get("XX", {})
<     velocity_thresh = tn.get("velocity_threshold", 0.02) # normalised shuttle
displacement/sec
---
>     w = idx_cfg.get("XX", {})
>     velocity_thresh = w.get("velocity_threshold", 0.02)
112,113c103
<         "weights_path": tn_cfg.weights_path,
<         "queue_length": tn_cfg.queue_length,
---
>         "weights_path": XX_cfg.weights_path,
116d105

```

30. user (2026-06-10T06:01:58.168Z)

```

Exit code 2
49:@app.get("/", response_class=HTMLResponse)
50:def serve_index():
63:def load_config(config_path: str = "config.json") -> Dict[str, Any]:
87:@app.get("/api/config")
88:def get_config():
91:@app.post("/api/process")
92:def process_video(req: ProcessRequest):
172:@app.post("/api/query")
173:def query_play_segments(req: QueryRequest):
183:@app.post("/api/compile")
184:def compile_highlights(req: CompileRequest):
209:def run_cli_diagnose_clip(video_path: str, timestamp: float):
280:def main():
===
2:import subprocess
10:     def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float =
0.5):
17:     def _parse_time(val: Union[int, float, str]) -> float:
46:     def _get_video_duration(self, video_path: str) -> float:
50:         result = subprocess.run(
64:     def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float, float]],
output_filename: str) -> str:
138:         "ffmpeg", "-y",
151:         result = subprocess.run(cmd, capture_output=True, check=True)
158:         except subprocess.CalledProcessError as e:
163:             skipped_segments.append((idx, start, end, f"ffmpeg error:
{stderr_msg[-200:]}"))
178:         f"Common causes: video file is corrupted, segments are out of range, or
ffmpeg is not installed."
194:         "ffmpeg", "-y",
203:         result = subprocess.run(concat_cmd, capture_output=True, check=True)
215:         except subprocess.CalledProcessError as e:
===
ls: cannot access 'tests/conftest.py': No such file or directory

```

31. user (2026-06-10T06:02:04.831Z)

```
1      tests/test_gemini.py::test_gemini_segmenter_chunked
2      C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\unittest\mock.py:2253:
ResourceWarning: unclosed database in <sqlite3.Connection object at 0x000001FF3F84C310>
3      def __init__(self, name, parent):
4          Enable tracemalloc to get traceback where the object was allocated.
5          See https://docs.pytest.org/en/stable/how-to/capture-warnings.html#resource-warnings
for more info.
6
7      -- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
8      ===== tests coverage =====
9      _____ coverage: platform win32, python 3.13.13-final-0 _____
10
11      Name                                                    Stmts   Miss  Cover    Missing
12      -----
13      backend\annotations\__init__.py                        4       0   100%
14      backend\annotations\automated_provider.py              33       0   100%
15      backend\annotations\base.py                            42       2    95%    91-93
16      backend\annotations\file_provider.py                   26       1    96%     49
17      backend\config\__init__.py                             3       0   100%
18      backend\config\loader.py                               29       3    90%    48-51, 75
19      backend\config\models.py                              91       4    96%   141-145
20      backend\eval\__init__.py                               0       0   100%
21      backend\eval\annotations.py                           47       2    96%    68-69
22      backend\eval\audio\__init__.py                       0       0   100%
23      backend\eval\audio\e1_probe.py                        67      40    40%    32, 41, 50,
55-90, 99-110, 115
24      backend\eval\batch.py                                  50       6    88%    81, 105-120
25      backend\eval\calibrate_local.py                       66      40    39%    67-74, 78-110,
114-122, 126
26      backend\eval\calibrate_wasb.py                        63      10    84%    78, 104-111, 115
27      backend\eval\calibration.py                          66       0   100%
28      backend\eval\classifier.py                            70       1    99%     61
29      backend\eval\distill_local.py                         92      54    41%    73-78, 82-86, 93,
100-141, 145-154, 158
30      backend\eval\eval_partial_labels.py                  104      20    81%    62, 74, 155,
191-211, 217
31      backend\eval\export_wasb_segments.py                  126      28    78%   174, 193-195,
199-226, 230
32      backend\eval\gemini_refine.py                        146     108    26%    76-98, 105-145,
156-196, 200-227, 233
33      backend\eval\harness.py                              68       1    99%    137
34      backend\eval\metrics.py                              76       0   100%
35      backend\eval\rally_seq_proto.py                      138       9    93%    78, 120, 144,
201-205, 209
36      backend\eval\regression.py                           69       3    96%    50, 112-113
37      backend\eval\training.py                             16       0   100%
38      backend\infrastructure\database.py                   135      21    84%   112-114, 126-128,
139-141, 162-164, 176-178, 226-227, 234-235, 239-240
39      backend\pipeline\audio\__init__.py                   0       0   100%
40      backend\pipeline\audio\hit_detector.py                99      32    68%    31-37, 42-55, 60,
74, 94, 114, 129, 138-146
41      backend\pipeline\detectors\__init__.py                1       0   100%
42      backend\pipeline\detectors\base.py                    9       2    78%    79, 87
43      backend\pipeline\detectors\rally_gate.py              59       2    97%    40, 49
44      backend\pipeline\detectors\tracknet_runner.py         153      18    88%    72, 97-100,
119-120, 161, 178-180, 202-204, 226-227, 252, 254
45      backend\pipeline\detectors\wasb_infer.py              247     156    37%    57-66, 71-73,
126-127, 169, 211-345, 356-376, 380-403, 410
46      backend\pipeline\detectors\wasb_runner.py             93      55    41%    44-45, 63-66,
76-79, 82, 86-97, 101-104, 108-146
47      backend\pipeline\segmenters\__init__.py               7       0   100%
48      backend\pipeline\segmenters\base.py                   42       7    83%    15, 21, 33,
44-45, 58-59
```

49	backend\pipeline\segmenters\gemini.py	265	81	69%	23, 34-36, 47-52,
56-58,					69-93, 100-103, 110, 147-148, 171-172, 176-177, 230-231, 263, 276-297, 309-310, 328-329,
333-334,					340-341, 345, 361-368, 375-376, 408-409
50	backend\pipeline\segmenters\motion.py	127	109	14%	20, 27-226
51	backend\pipeline\segmenters\tracknet_hybrid.py	154	16	90%	65-67, 84-86,
90-91,					172, 186, 214-215, 231-232, 247-248
52	backend\pipeline\segmenters\wasb_hybrid.py	151	127	16%	32-35, 45, 50-54,
58-222					
53	backend\pipeline\segmenters\yolo_hybrid.py	314	78	75%	64, 73-91,
98-100,					127, 137, 162-163, 167, 171, 195-196, 230, 253-254, 270-272, 292-294, 300-301, 349-388,
399,					416-418, 433-437, 475-476, 494, 526-527, 543, 566-567
54	backend\pipeline\validators__init__.py	14	0	100%	
55	backend\pipeline\validators\base.py	39	3	92%	16, 22, 32
56	backend\pipeline\validators\blur.py	40	31	22%	8, 12, 15-79
57	backend\pipeline\validators\format.py	32	21	34%	10, 14, 17-82
58	backend\pipeline\validators\pts_continuity.py	51	42	18%	8, 12, 16-112
59	backend\pipeline\validators\relevance.py	56	46	18%	9, 13, 16-105
60	backend\pipeline\validators\resolution_fps.py	32	24	25%	7, 11, 14-62
61	backend\pipeline\validators\scene_cut.py	53	43	19%	9, 13, 16-97
62	backend\providers__init__.py	16	1	94%	16
63	backend\providers\base.py	9	1	89%	27
64	backend\providers\gemini.py	96	26	73%	71-73, 75-87,
110-113,					116-117, 120, 129-131
65	backend\reporting__init__.py	25	2	92%	47-48
66	backend\reporting\harvest.py	111	8	93%	19-20, 38-39,
56-57,					63-64
67	backend\results.py	22	0	100%	
68	backend\sports__init__.py	17	1	94%	17
69	backend\sports\badminton.py	13	1	92%	63
70	backend\sports\base.py	12	3	75%	8, 13, 18
71	backend\tools__init__.py	0	0	100%	
72	backend\tools\export.py	46	0	100%	
73	backend\tools\rally_slicer.py	104	47	55%	48-49, 67, 101,
107-115,					122-155, 159-172, 176
74	backend\utils__init__.py	0	0	100%	
75	backend\utils\geometry.py	29	0	100%	
76	backend\utils\hashing.py	9	0	100%	
77	backend\utils\video.py	26	3	88%	41-43
78	-----				
79	TOTAL	4200	1339	68%	
80	256 passed, 115 warnings in 36.19s				
81					

32. user (2026-06-10T06:02:31.197Z)

```

1 import pytest
2 import sqlite3
3 import os
4 from backend.infrastructure.database import Database
5
6 def test_database_init(tmp_path):
7     db_path = tmp_path / "test.db"
8     db = Database(str(db_path))
9
10    conn = sqlite3.connect(str(db_path))
11    cursor = conn.cursor()
12    cursor.execute("SELECT name FROM sqlite_master WHERE type='table'")
13    tables = [r[0] for r in cursor.fetchall()]
14    assert "videos" in tables
15    assert "play_segments" in tables
16    conn.close()
17
18 def test_database_video(tmp_path):
19     db = Database(str(tmp_path / "test.db"))
20

```

```

21     vid_added = db.add_video("vid1", "vid1.mp4", "pending", [{"val": 1}])
22     assert vid_added is True
23
24     video = db.get_video("vid1")
25     assert video["id"] == "vid1"
26     assert video["filepath"] == "vid1.mp4"
27     assert video["status"] == "pending"
28     assert video["validation_results"] == [{"val": 1}]
29
30     db.clear_video_data("vid1")
31     assert db.get_video("vid1") is None
32
33 def test_database_play_segment(tmp_path):
34     db = Database(str(tmp_path / "test.db"))
35     db.add_video("vid1", "vid1.mp4", "pending", [])
36
37     sid = db.add_play_segment("vid1", 10.0, 20.0, "PlayerA", "PlayerB", {"foo": "bar"})
38     assert sid is not None
39
40     db.add_event(sid, 15.0, "smash", "PlayerA", {"speed": 300})
41
42     segments = db.query_play_segments("vid1", {"server": "PlayerA"})
43     assert len(segments) == 1
44     assert segments[0]["id"] == sid
45     assert segments[0]["start_time"] == 10.0
46     assert segments[0]["server"] == "PlayerA"
47     assert segments[0]["metadata"]["foo"] == "bar"
48
49     assert len(segments[0]["events"]) == 1
50     assert segments[0]["events"][0]["type"] == "smash"
51     assert segments[0]["events"][0]["details"]["speed"] == 300
52
53
54 # --- candidate_segments schema + behavior -----
55
56 # The schema contract: any future change to candidate_segments columns must
57 # consciously update this set, otherwise the test below fails (regression guard).
58 EXPECTED_CANDIDATE_COLUMNS = {
59     "id", "video_id", "detector", "chunk_index", "start_time", "end_time",
60     "duration", "confidence", "stats", "detection_params",
61     "confirmed_segment_id", "human_label", "notes", "created_at",
62 }
63
64
65 def test_candidate_schema_contract(tmp_path):
66     db = Database(str(tmp_path / "test.db"))
67     conn = sqlite3.connect(str(tmp_path / "test.db"))
68     cols = conn.execute("PRAGMA table_info(candidate_segments)").fetchall()
69     conn.close()
70
71     names = {c[1] for c in cols}
72     assert names == EXPECTED_CANDIDATE_COLUMNS
73
74     # NOT NULL / PK flags on the load-bearing columns (cid, name, type, notnull, dflt,
75 pk)
76     by_name = {c[1]: c for c in cols}
77     assert by_name["id"][5] == 1 # primary key
78     for col in ("video_id", "detector", "start_time", "end_time", "duration"):
79         assert by_name[col][3] == 1, f"{col} should be NOT NULL"
80
81 def test_schema_version_is_current(tmp_path):
82     db_path = tmp_path / "test.db"
83     Database(str(db_path))
84     conn = sqlite3.connect(str(db_path))

```

```

85     version = conn.execute("PRAGMA user_version").fetchone()[0]
86     conn.close()
87     assert version == 1
88
89
90 def test_init_db_idempotent_and_upgrades_old_db(tmp_path):
91     db_path = tmp_path / "legacy.db"
92     # Simulate a pre-v1 database that only has the old tables and no version.
93     conn = sqlite3.connect(str(db_path))
94     conn.execute("CREATE TABLE videos (id TEXT PRIMARY KEY, filepath TEXT, status
TEXT)")
95     conn.commit()
96     conn.close()
97
98     # Opening with the new Database should add candidate_segments without error...
99     Database(str(db_path))
100    # ...and running init again must be a no-op (idempotent).
101    Database(str(db_path))
102
103    conn = sqlite3.connect(str(db_path))
104    tables = [r[0] for r in conn.execute(
105        "SELECT name FROM sqlite_master WHERE type='table'").fetchall()]
106    version = conn.execute("PRAGMA user_version").fetchone()[0]
107    conn.close()
108    assert "candidate_segments" in tables
109    assert version == 1
110
111
112 def test_candidate_round_trip(tmp_path):
113     db = Database(str(tmp_path / "test.db"))
114     db.add_video("vid1", "vid1.mp4", "processed", [])
115
116     cid = db.add_candidate_segment(
117         "vid1", "yolo_hybrid", 10.0, 25.0, chunk_index=0, confidence=0.42,
118         stats={"peak_velocity": 0.8, "track_id_count": 2},
119         detection_params={"frame_skip": 3},
120     )
121     assert cid is not None
122
123     cands = db.query_candidate_segments("vid1")
124     assert len(cands) == 1
125     c = cands[0]
126     assert c["detector"] == "yolo_hybrid"
127     assert c["start_time"] == 10.0
128     assert c["duration"] == 15.0
129     assert c["confidence"] == 0.42
130     assert c["stats"]["peak_velocity"] == 0.8
131     assert c["detection_params"]["frame_skip"] == 3
132     assert c["confirmed_segment_id"] is None
133
134     # detector filter
135     assert db.query_candidate_segments("vid1", detector="motion") == []
136     assert len(db.query_candidate_segments("vid1", detector="yolo_hybrid")) == 1
137
138
139 def test_candidate_link_and_unlabeled_filter(tmp_path):
140     db = Database(str(tmp_path / "test.db"))
141     db.add_video("vid1", "vid1.mp4", "processed", [])
142
143     cid = db.add_candidate_segment("vid1", "yolo_hybrid", 1.0, 4.0)
144     sid = db.add_play_segment("vid1", 1.0, 4.0)
145
146     assert db.link_candidate_to_segment(cid, sid) is True
147     assert db.query_candidate_segments("vid1")[0]["confirmed_segment_id"] == sid
148

```



```

149     # only_unlabeled returns it while human_label is NULL
150     assert len(db.query_candidate_segments("vid1", only_unlabeled=True)) == 1
151     with db._get_connection() as conn:
152         conn.execute("UPDATE candidate_segments SET human_label = ? WHERE id = ?",
153                     ("true_rally", cid))
154         conn.commit()
155     assert db.query_candidate_segments("vid1", only_unlabeled=True) == []
156
157
158     def test_candidate_cascade_on_video_delete(tmp_path):
159         db = Database(str(tmp_path / "test.db"))
160         db.add_video("vid1", "vid1.mp4", "processed", [])
161         db.add_candidate_segment("vid1", "yolo_hybrid", 1.0, 4.0)
162
163         db.clear_video_data("vid1")
164         assert db.query_candidate_segments("vid1") == []
165

```

33. user (2026-06-10T06:02:31.679Z)

```

1     """Golden guard: every footgun rule's faq_ref must resolve to a real README heading.
2
3     This is the committed backbone of report debuggability ? if a rule cites README#Qn,
4     that section must exist, so a reader who follows the pointer always lands somewhere.
5     (A fresh-reader eval found a rule citing the wrong-but-existing section; an existence
6     test can't catch *semantic* mismatch, but it locks the floor: no dangling anchors.)
7     """
8     import re
9     from pathlib import Path
10
11     import pytest
12
13     pytest.importorskip("obsreport")
14     from obsreport import load_spec # noqa: E402
15
16     ROOT = Path(__file__).resolve().parent.parent
17     SPEC = ROOT / "backend" / "reporting" / "spec.yaml"
18     README = ROOT / "README.md"
19
20
21     def _headings(md: str):
22         return [ln.lstrip("#").strip() for ln in md.splitlines() if
23                 ln.lstrip().startswith("#")]
24
25
26     def _slug(h: str) -> str:
27         s = re.sub(r"^[^\w\s-]", "", h.lower())
28         return re.sub(r"\s+", "-", s.strip())
29
30
31     def _anchor_resolves(anchor: str, headings) -> bool:
32         a = anchor.lower()
33         for h in headings:
34             hl = h.lower()
35             if hl == a or hl.startswith(a + ":") or hl.startswith(a + " ") or
36                 hl.startswith(a + "."):
37                 return True # Q-number style: "Q7" -> "### Q7: ..."
38             if _slug(h) == a:
39                 return True # full GitHub slug match
40         return False
41
42
43     def test_every_rule_faq_ref_resolves_to_a_readme_heading():
44         spec = load_spec(SPEC)
45         headings = _headings(README.read_text(encoding="utf-8"))

```

```

44     unresolved = []
45     for rule in spec.rules:
46         if not rule.faq_ref:
47             continue
48         file_part, _, anchor = rule.faq_ref.partition("#")
49         if file_part in ("README", "README.md"):
50             if not _anchor_resolves(anchor, headings):
51                 unresolved.append((rule.code, rule.faq_ref))
52         elif file_part in ("VIDEO_RECORDING_GUIDELINES",
53 "VIDEO_RECORDING_GUIDELINES.md",
54 "AUDIO_RALLY_DETECTION_PLAN",
55 "AUDIO_RALLY_DETECTION_PLAN.md"):
56             # These are dedicated guidance docs surfaced from report rules (Input
57             # extension). The whole document is the destination; no #Q anchor required.
58             pass
59         else:
60             unresolved.append((rule.code, rule.faq_ref))
61     assert not unresolved, f"faq_ref anchors with no matching README heading:
62 {unresolved}"
63
64 def test_actionable_rules_have_faq_refs():
65     """error/warn findings must cite a FAQ (info may omit). Belt-and-suspenders on top
66     of obsreport's own spec-load validation."""
67     spec = load_spec(SPEC)
68     missing = [r.code for r in spec.rules if r.severity in ("error", "warn") and not
69 r.faq_ref]
70     assert not missing, f"actionable rules missing faq_ref: {missing}"

```

34. user (2026-06-10T06:02:36.507Z)

```

1     """Tests for the regression runner (GS-2): baseline store, regress(), the
2     provider-driven end-to-end path, and the CLI exit-code contract."""
3     import json
4     import pytest
5
6     from backend.infrastructure.database import Database
7     from backend.eval import regression
8     import run_regression
9
10
11     @pytest.fixture
12     def db(tmp_path):
13         d = Database(str(tmp_path / "t.db"))
14         d.add_video("vid1", "/x/m.mp4", "processed", [])
15         # Two confirmed segments matching the GT below ? perfect baseline.
16         d.add_play_segment("vid1", 0.0, 10.0)
17         d.add_play_segment("vid1", 20.0, 30.0)
18         return d
19
20
21     GT = [(0.0, 10.0), (20.0, 30.0)]
22
23
24     # ----- baseline store -----
25
26     def test_save_and_load_baseline_roundtrip(tmp_path):
27         p = str(tmp_path / "b.json")
28         regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
29         b = regression.load_baselines(p)
30         assert b["vid1::final"]["precision"] == 1.0
31
32

```

```

33 def test_load_baselines_absent_is_empty(tmp_path):
34     assert regression.load_baselines(str(tmp_path / "nope.json")) == {}
35
36
37 # ----- regress() -----
38
39 def test_no_baseline_is_not_a_regression(db, tmp_path):
40     r = regression.regress(db, "vid1", GT, baseline_path=str(tmp_path / "b.json"))
41     assert r.regressed is False
42     assert r.precision == 1.0 and r.recall == 1.0
43     assert any("no baseline" in why for why in r.reasons)
44
45
46 def test_matching_baseline_passes(db, tmp_path):
47     p = str(tmp_path / "b.json")
48     regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
49     r = regression.regress(db, "vid1", GT, baseline_path=p)
50     assert r.regressed is False
51
52
53 def test_recall_drop_beyond_tolerance_regresses(tmp_path):
54     # DB has only ONE of the two GT rallies ? recall 0.5 vs baseline 1.0.
55     d = Database(str(tmp_path / "t.db"))
56     d.add_video("vid1", "/x/m.mp4", "processed", [])
57     d.add_play_segment("vid1", 0.0, 10.0)
58     p = str(tmp_path / "b.json")
59     regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
60
61     r = regression.regress(d, "vid1", GT, tolerance=0.05, baseline_path=p)
62     assert r.regressed is True
63     assert r.recall == pytest.approx(0.5)
64     assert any("recall" in why for why in r.reasons)
65
66
67 def test_within_tolerance_does_not_regress(tmp_path):
68     d = Database(str(tmp_path / "t.db"))
69     d.add_video("vid1", "/x/m.mp4", "processed", [])
70     d.add_play_segment("vid1", 0.0, 10.0)
71     d.add_play_segment("vid1", 20.0, 30.0)
72     p = str(tmp_path / "b.json")
73     # Baseline only 0.02 above current perfect score ? within default 0.05 tolerance.
74     regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=p)
75     r = regression.regress(d, "vid1", GT, tolerance=0.05, baseline_path=p)
76     assert r.regressed is False
77
78
79 # ----- provider-driven end-to-end (uses the GS-1 FileProvider) -----
80
81 def test_regress_with_provider_file_tier(db, tmp_path):
82     csv = tmp_path / "gt.csv"
83     csv.write_text("start,end\n0,10\n20,30\n")
84     r = regression.regress_with_provider(
85         db, video_id="vid1", video_ref=str(csv), tier="file",
86         baseline_path=str(tmp_path / "b.json"),
87     )
88     assert r.precision == 1.0 and r.recall == 1.0
89
90
91 # ----- CLI exit-code contract (CI gating) -----
92
93 def test_cli_update_baseline_then_clean_run(db, tmp_path, capsys, monkeypatch):
94     # Point the runner at our temp DB via --db; CSV ground truth via 'file' tier.
95     csv = tmp_path / "gt.csv"
96     csv.write_text("start,end\n0,10\n20,30\n")
97     base = str(tmp_path / "b.json")

```

```

98         db_path = db.db_path if hasattr(db, "db_path") else str(tmp_path / "t.db")
99
100        # First, snapshot the baseline ? exit 0.
101        rc = run_regression.main(["--video-id", "vid1", "--tier", "file",
102                                "--annotations", str(csv), "--db", db_path,
103                                "--baseline", base, "--update-baseline"])
104        assert rc == 0
105        assert "REGRESSED" not in capsys.readouterr().out
106
107
108    def test_cli_regression_exits_nonzero(tmp_path, capsys):
109        # DB missing one rally vs a baseline of perfect ? regression ? exit 1.
110        d = Database(str(tmp_path / "t.db"))
111        d.add_video("vid1", "/x/m.mp4", "processed", [])
112        d.add_play_segment("vid1", 0.0, 10.0)
113        csv = tmp_path / "gt.csv"
114        csv.write_text("start,end\n0,10\n20,30\n")
115        base = str(tmp_path / "b.json")
116        regression.save_baseline("vid1", "final", 1.0, 1.0, 1.0, path=base)
117
118        rc = run_regression.main(["--video-id", "vid1", "--tier", "file",
119                                "--annotations", str(csv), "--db", str(tmp_path / "t.db"),
120                                "--baseline", base])
121        assert rc == 1
122        assert "REGRESSED" in capsys.readouterr().out
123

```

35. user (2026-06-10T06:02:36.883Z)

```

1    import pytest
2    from unittest.mock import MagicMock, patch
3    from backend.providers.gemini import GeminiProvider
4
5    def test_gemini_provider_success():
6        # Mock GenAI client
7        with patch("backend.providers.gemini.genai") as mock_genai:
8            client_mock = MagicMock()
9            mock_genai.Client.return_value = client_mock
10
11            # Mock file upload
12            file_mock = MagicMock()
13            file_mock.uri = "gs://dummy"
14            file_mock.name = "dummy_file"
15            file_mock.state.name = "ACTIVE"
16
17            client_mock.files.upload.return_value = file_mock
18            client_mock.files.get.return_value = file_mock
19
20            # Mock generate_content response
21            response_mock = MagicMock()
22            response_mock.text = ' [{"start_time": 1.0, "end_time": 2.0}] '
23            client_mock.models.generate_content.return_value = response_mock
24
25            provider = GeminiProvider(api_key="dummy_api_key",
26                                     model_name="gemini-2.5-flash")
27            segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt",
28                                                       chunk_duration=10.0)
29
30            assert len(segments) == 1
31            assert segments[0]["start_time"] == 1.0
32            assert metrics["upload_time"] >= 0
33            assert metrics["process_time"] >= 0
34            assert metrics["gen_time"] >= 0
35
36            # Ensure delete is called
37

```

```

35         client_mock.files.delete.assert_called_with(name="dummy_file")
36
37     def test_gemini_provider_invalid_json():
38         with patch("backend.providers.gemini.genai") as mock_genai:
39             client_mock = MagicMock()
40             mock_genai.Client.return_value = client_mock
41             file_mock = MagicMock()
42             file_mock.state.name = "ACTIVE"
43             client_mock.files.upload.return_value = file_mock
44             client_mock.files.get.return_value = file_mock
45
46             response_mock = MagicMock()
47             response_mock.text = '{"not_a_list": true}'
48             client_mock.models.generate_content.return_value = response_mock
49
50             provider = GeminiProvider(api_key="dummy_api_key",
model_name="gemini-2.5-flash")
51             segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt")
52             assert len(segments) == 0
53
54     def test_gemini_provider_rejects_oversize_before_upload(monkeypatch):
55         with patch("backend.providers.gemini.genai") as mock_genai:
56             client_mock = MagicMock()
57             mock_genai.Client.return_value = client_mock
58             # 600MB > MAX_UPLOAD_MB (500) -> refuse without ever uploading
59             monkeypatch.setattr("backend.providers.gemini.os.path.getsize",
60                                 lambda p: 600 * 1024 * 1024)
61             provider = GeminiProvider(api_key="k", model_name="gemini-flash-latest")
62             segments, metrics = provider.analyze_video("big.mp4", "p")
63             assert segments == []
64             client_mock.files.upload.assert_not_called()
65
66
67     def test_gemini_provider_upload_fail():
68         with patch("backend.providers.gemini.genai") as mock_genai:
69             client_mock = MagicMock()
70             mock_genai.Client.return_value = client_mock
71             client_mock.files.upload.side_effect = Exception("Upload err")
72
73             provider = GeminiProvider(api_key="dummy_api_key",
model_name="gemini-2.5-flash")
74             segments, metrics = provider.analyze_video("dummy.mp4", "dummy prompt")
75             assert len(segments) == 0
76

```

36. user (2026-06-10T06:02:44.306Z)

```

1     """Tests for the validation registry, focused on the backward-compatible
2     `run_all_with_context` added for observability reporting."""
3     from backend.pipeline.validators.base import ValidationRegistry, ValidationResult,
VideoValidator
4
5
6     class _ProducerValidator(VideoValidator):
7         @property
8         def name(self):
9             return "producer"
10
11         @property
12         def description(self):
13             return "writes ffprobe_meta into the shared context"
14
15         def validate(self, video_path, config, context):
16             context["ffprobe_meta"] = {"fps": 59.94, "width": 1920, "height": 1080}
17             return ValidationResult(validator_name=self.name, passed=True, message="ok")

```

```

18
19
20 class _ConsumerValidator(VideoValidator):
21     @property
22     def name(self):
23         return "consumer"
24
25     @property
26     def description(self):
27         return "reads ffprobe_meta the producer cached"
28
29     def validate(self, video_path, config, context):
30         seen = "ffprobe_meta" in context
31         return ValidationResult(validator_name=self.name, passed=seen,
message=f"saw_meta={seen}")
32
33
34 def _registry():
35     r = ValidationRegistry()
36     r.register(_ProducerValidator())
37     r.register(_ConsumerValidator())
38     return r
39
40
41 def test_run_all_with_context_returns_results_and_context():
42     results, context = _registry().run_all_with_context("v.mp4", {})
43     assert [r.validator_name for r in results] == ["producer", "consumer"]
44     assert context["ffprobe_meta"]["fps"] == 59.94
45     assert results[1].passed is True # consumer saw the shared context
46
47
48 def test_run_all_is_unchanged_and_returns_only_results():
49     results = _registry().run_all("v.mp4", {})
50     assert isinstance(results, list)
51     assert all(isinstance(r, ValidationResult) for r in results)
52     assert [r.validator_name for r in results] == ["producer", "consumer"]
53
54
55 def test_validator_exception_is_captured_not_raised():
56     class _Boom(VideoValidator):
57         @property
58         def name(self):
59             return "boom"
60
61         @property
62         def description(self):
63             return "raises"
64
65         def validate(self, video_path, config, context):
66             raise RuntimeError("explode")
67
68     r = ValidationRegistry()
69     r.register(_Boom())
70     results, _ = r.run_all_with_context("v.mp4", {})
71     assert results[0].passed is False
72     assert "explode" in results[0].message
73

```

37. user (2026-06-10T06:02:44.742Z)

```

1     import pytest
2
3     from backend.annotations import annotation_registry, AnnotationProvider
4     from backend.annotations.base import DONE, FAILED
5     from backend.annotations.file_provider import FileProvider

```

```

6
7
8 def _write_csv(path) -> str:
9     # Mixed decimal + HH:MM:SS forms; header auto-skipped by load_annotations.
10    path.write_text("start,end\n0.0,3.0\n00:00:05,00:00:09\n")
11    return str(path)
12
13
14 # ----- registry -----
15
16 def test_registry_has_file_provider():
17     assert "file" in annotation_registry.list_available()
18     p = annotation_registry.get("file")
19     assert isinstance(p, AnnotationProvider)
20     assert p.name == "file"
21
22
23 def test_registry_unknown_provider_raises():
24     with pytest.raises(KeyError):
25         annotation_registry.get("does-not-exist")
26
27
28 def test_registry_get_passes_kwargs(tmp_path):
29     p = annotation_registry.get("file", csv_dir=str(tmp_path))
30     assert isinstance(p, FileProvider)
31     assert p.csv_dir == str(tmp_path)
32
33
34 # ----- FileProvider: submit / poll / fetch -----
35
36 def test_file_provider_direct_path(tmp_path):
37     csv = _write_csv(tmp_path / "rallies.csv")
38     p = FileProvider()
39     job = p.submit(csv)
40     assert job == csv
41     assert p.poll(job) == DONE
42     assert p.fetch(job) == [(0.0, 3.0), (5.0, 9.0)]
43
44
45 def test_file_provider_dir_resolution(tmp_path):
46     _write_csv(tmp_path / "vid123.csv")
47     p = FileProvider(csv_dir=str(tmp_path))
48     job = p.submit("vid123") # bare video_id, resolved to <dir>/vid123.csv
49     assert job.endswith("vid123.csv")
50     assert p.poll(job) == DONE
51     assert len(p.fetch(job)) == 2
52
53
54 def test_file_provider_missing_file_is_failed(tmp_path):
55     p = FileProvider(csv_dir=str(tmp_path))
56     job = p.submit("nonexistent")
57     assert p.poll(job) == FAILED
58     with pytest.raises(FileNotFoundError):
59         p.fetch(job)
60
61
62 def test_file_provider_annotate_convenience(tmp_path):
63     csv = _write_csv(tmp_path / "rallies.csv")
64     p = FileProvider()
65     assert p.annotate(csv) == [(0.0, 3.0), (5.0, 9.0)]
66
67
68 def test_annotate_raises_when_not_done(tmp_path):
69     # annotate() on a synchronous provider with a missing file must fail loudly,
70     # not silently return [] (which would skew downstream regression scores).

```

```

71     p = FileProvider(csv_dir=str(tmp_path))
72     with pytest.raises((RuntimeError, FileNotFoundError)):
73         p.annotate("nonexistent")
74
75
76     # ----- the contract that makes the whole flow work: provider output == GT shape -----
77
78     def test_provider_output_matches_metrics_interval_shape(tmp_path):
79         """A provider's fetch() output must be directly scorable by the eval harness ?
80         same List[(float, float)] shape metrics.score() expects. This is the keystone
81         that lets one scorer grade product, vendor, and annotator-vs-annotator."""
82         from backend.eval import metrics
83         csv = _write_csv(tmp_path / "gt.csv")
84         gt = FileProvider().annotate(csv)
85         # Feed provider output straight into the scorer as ground truth ? must not raise.
86         s = metrics.score(preds=[(0.0, 3.0), (5.0, 9.0)], gts=gt, iou_threshold=0.5)
87         assert s.true_positives == 2
88         assert s.precision == 1.0 and s.recall == 1.0
89

```

38. user (2026-06-10T06:02:44.959Z)

```

1     """Basic tests for the typed configuration package.
2
3     These ensure:
4     - Defaults are sensible and come from the models.
5     - File overrides work.
6     - Old config files with extra keys are tolerated.
7     - save_default_config produces loadable output.
8     """
9
10    import json
11    import tempfile
12    import os
13
14    import pytest
15
16    from backend.config import load_config, save_default_config, AppConfig
17
18
19    def test_load_default_config():
20        # Force load without the project's config.json so we test the model built-in
21        defaults
22        cfg = load_config("/nonexistent/path/that/does/not/exist/12345/config.json")
23        assert isinstance(cfg, AppConfig)
24        assert cfg.sport == "badminton"
25        assert cfg.indexing.default_segmenter == "yolo_hybrid"
26        assert cfg.validation.min_blur_threshold == 20.0 # updated default from review
27        assert cfg.indexing.ai_max_workers == 5
28
29    def test_file_override():
30        with tempfile.TemporaryDirectory() as td:
31            p = os.path.join(td, "override.json")
32            data = {
33                "indexing": {
34                    "default_segmenter": "gemini",
35                    "ai_max_workers": 3,
36                },
37                "validation": {
38                    "min_blur_threshold": 42.0,
39                },
40            }
41            with open(p, "w") as f:
42                json.dump(data, f)

```



```

43
44     cfg = load_config(p)
45     assert cfg.indexing.default_segmenter == "gemini"
46     assert cfg.indexing.ai_max_workers == 3
47     assert cfg.validation.min_blur_threshold == 42.0
48     # Unspecified keys fall back to defaults
49     assert cfg.sport == "badminton"
50
51
52 def test_extra_keys_tolerated():
53     """Old or future keys in config.json must not break loading."""
54     with tempfile.TemporaryDirectory() as td:
55         p = os.path.join(td, "with_extra.json")
56         data = {
57             "sport": "badminton",
58             "indexing": {"default_segmenter": "yolo_hybrid"},
59             "some_future_key": "ignored_for_now",
60             "validation": {"relevance": {"court_pixels_min_ratio": 0.1}},
61         }
62         with open(p, "w") as f:
63             json.dump(data, f)
64
65         cfg = load_config(p)
66         assert cfg.indexing.default_segmenter == "yolo_hybrid"
67         assert cfg.validation.relevance.court_pixels_min_ratio == 0.1
68
69
70 def test_save_default_config_roundtrip():
71     with tempfile.TemporaryDirectory() as td:
72         p = os.path.join(td, "saved.json")
73         saved = save_default_config(p)
74         assert str(saved) == p
75         assert os.path.exists(p)
76
77         cfg = load_config(p)
78         assert cfg.indexing.default_segmenter == "yolo_hybrid"
79         assert cfg.validation.min_blur_threshold == 20.0
80
81
82 def test_model_is_the_source_of_truth():
83     """Changing a default in the model should be reflected without touching json."""
84     # This is a documentation test more than anything.
85     cfg = AppConfig()
86     assert cfg.indexing.tracknet.velocity_threshold == 0.02
87
88
89 def test_output_dir_field_default_and_override():
90     """output_dir now lives on OutputConfig (was a write-only runtime hack), so
91     /api/compile and the CLI resolve the same value. Default + file override + the
92     runtime mutation that backend/main.py::main performs for --output-dir."""
93     cfg = AppConfig()
94     assert cfg.output.output_dir == "output" # built-in default
95
96     # The CLI override path: main() assigns args.output_dir onto the typed model.
97     cfg.output.output_dir = "/tmp/custom_out"
98     assert cfg.output.output_dir == "/tmp/custom_out"
99
100     # File override is honored on load.
101     with tempfile.TemporaryDirectory() as td:
102         p = os.path.join(td, "out_override.json")
103         with open(p, "w") as f:
104             json.dump({"output": {"output_dir": "reels", "db_name": "x.db"}}, f)
105         loaded = load_config(p)
106         assert loaded.output.output_dir == "reels"
107         assert loaded.output.db_name == "x.db"

```

```

108
109
110 def test_get_returns_dict_for_nested_sections_legacy_compat():
111     """Regression: legacy consumers (validators/segmenters) do
112     ``config.get("validation", {}).get("min_resolution_width", ...)``. The dict-compat
113     .get must return nested SECTIONS as plain dicts (not Pydantic models), or that
114     chained access raises 'X object has no attribute get' on every real run."""
115     cfg = AppConfig()
116     validation = cfg.get("validation", {})
117     assert isinstance(validation, dict)
118     assert validation.get("min_resolution_width") == 1280
119     # nested-of-nested is also dict-accessible
120     assert isinstance(validation.get("relevance"), dict)
121     assert validation["relevance"].get("court_pixels_min_ratio") == 0.05
122     assert cfg.get("indexing", {}).get("default_segmenter") == "yolo_hybrid"
123     # scalar leaves are returned unchanged; a present scalar ignores the default arg;
124     # a missing key falls back to the default
125     assert cfg.get("ai_provider") == "gemini"
126     assert cfg.get("ai_provider", "ignored-default") == "gemini"
127     assert cfg.get("does_not_exist", "fallback") == "fallback"
128     # leaf keys inside any section resolve via the dynamic section scan (not a hardcoded
129     # list), so a future top-level section's keys would be discoverable too
130     assert cfg.get("min_resolution_width") == 1280 # lives inside `validation`
131     assert cfg.get("begin_padding") == 0.5 # lives inside `stitching`
132

```

39. user (2026-06-10T06:02:55.446Z)

```

1     """Tests for the TrackNet runner's pure-Python pieces (path translation, CSV
2     parsing, trajectory -> action-window conversion). The WSL subprocess invocation
3     itself is not exercised here ? these run anywhere, no WSL/TensorFlow needed."""
4
5     import os
6     import subprocess
7     import textwrap
8     from unittest.mock import patch
9
10    from backend.pipeline.detectors.tracknet_runner import (
11        TrackNetRunner,
12        TrackNetConfig,
13        TrajectoryPoint,
14        to_wsl_mnt_path,
15    )
16    from backend.pipeline.detectors.base import DetectorRunner
17    from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
18
19
20    def _proc(returncode=0, stdout="", stderr=""):
21        return subprocess.CompletedProcess(args=[], returncode=returncode,
22                                           stdout=stdout, stderr=stderr)
23
24
25    def test_to_wsl_mnt_path_windows():
26        assert to_wsl_mnt_path(r"C:\Users\avidu\v.mp4") == "/mnt/c/Users/avidu/v.mp4"
27        assert to_wsl_mnt_path(r"D:\clips\m.mp4") == "/mnt/d/clips/m.mp4"
28
29
30    def test_to_wsl_mnt_path_passthrough():
31        # Already-POSIX paths are left alone.
32        assert to_wsl_mnt_path("/home/avidullu/clips/v.mp4") == "/home/avidullu/clips/v.mp4"
33        assert to_wsl_mnt_path("~/clips/v.mp4") == "~/clips/v.mp4"
34
35
36    def test_config_from_indexing_cfg():
37        cfg = TrackNetConfig.from_indexing_cfg({"tracknet": {

```

```

38         "wsl_distro": "Ubuntu-22.04", "conda_env": "TN", "weights_path": "/w/m.h5",
39         "queue_length": 7, "stage_in_wsl": False,
40     })
41     assert cfg.distro == "Ubuntu-22.04"
42     assert cfg.conda_env == "TN"
43     assert cfg.weights_path == "/w/m.h5"
44     assert cfg.queue_length == 7
45     assert cfg.stage_in_wsl is False
46
47
48     def test_config_defaults_when_absent():
49         cfg = TrackNetConfig.from_indexing_cfg({})
50         assert cfg.distro == "Ubuntu"
51         assert cfg.conda_env == "TrackNetV4"
52         assert cfg.weights_path == "" # must be supplied by the user
53
54
55     def test_parse_trajectory_csv(tmp_path):
56         csv_file = tmp_path / "v_predict.csv"
57         csv_file.write_text(textwrap.dedent("""\
58             Frame,Visibility,X,Y
59             0,0,0,0
60             1,1,100,200
61             2,1,140,210
62             3,0,0,0
63         """))
64         pts = TrackNetRunner.parse_trajectory_csv(str(csv_file))
65         assert len(pts) == 4
66         assert pts[0].visible is False
67         assert pts[1].visible is True and pts[1].x == 100 and pts[1].y == 200
68         assert pts[3].visible is False
69
70
71     def test_trajectory_to_action_windows_detects_moving_shuttle():
72         # 30 fps, 1280px wide. Shuttle moves ~60px/frame while visible (frames 0..29) =>
73         # well above the velocity threshold; then disappears. Expect one window ~1s long.
74         fps, width = 30.0, 1280.0
75         pts = [TrajectoryPoint(frame=i, visible=True, x=100 + 60 * i, y=300) for i in
76 range(30)]
77         pts += [TrajectoryPoint(frame=i, visible=False, x=0, y=0) for i in range(30, 60)]
78
79         windows = TrackNetRunner.trajectory_to_action_windows(
80             pts, fps=fps, frame_width=width, velocity_thresh=0.02,
81             merge_gap=2.0, min_window_duration=0.5, chunk_index=0,
82         )
83         assert len(windows) == 1
84         w = windows[0]
85         assert w["start"] < w["end"]
86         assert w["active_frame_count"] > 0
87         assert w["peak_velocity"] > 0.02
88         assert w["chunk_index"] == 0
89
90     def test_trajectory_to_action_windows_ignores_stationary_shuttle():
91         # Visible but barely moving (1px/frame) => below threshold => no windows.
92         pts = [TrajectoryPoint(frame=i, visible=True, x=500 + i, y=300) for i in range(60)]
93         windows = TrackNetRunner.trajectory_to_action_windows(
94             pts, fps=30.0, frame_width=1280.0, velocity_thresh=0.05,
95             merge_gap=2.0, min_window_duration=0.5,
96         )
97         assert windows == []
98
99
100     def test_trajectory_to_action_windows_splits_on_gap():
101         # Two bursts of motion separated by a long invisible gap => two windows.

```

```

102     burst1 = [TrajectoryPoint(frame=i, visible=True, x=100 + 60 * i, y=300) for i in
range(20)]
103     gap = [TrajectoryPoint(frame=i, visible=False, x=0, y=0) for i in range(20, 140)]
104     burst2 = [TrajectoryPoint(frame=i, visible=True, x=100 + 60 * (i - 140), y=300)
105               for i in range(140, 160)]
106     windows = TrackNetRunner.trajectory_to_action_windows(
107         burst1 + gap + burst2, fps=30.0, frame_width=1280.0,
108         velocity_thresh=0.02, merge_gap=2.0, min_window_duration=0.3,
109     )
110     assert len(windows) == 2
111     assert windows[0]["end"] < windows[1]["start"]
112
113
114     # ----- WSL subprocess glue
115
116     def _runner():
117         return TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5"))
118
119
120     def test_healthcheck_ok():
121         r = _runner()
122         with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="TF 2.17.0 GPUs 1")):
123             ok, msg = r.healthcheck()
124             assert ok is True
125             assert "TF 2.17.0" in msg
126
127
128     def test_healthcheck_failure_returns_message():
129         r = _runner()
130         with patch.object(r, "_wsl_bash", return_value=_proc(1,
stderr="ModuleNotFoundError")):
131             ok, msg = r.healthcheck()
132             assert ok is False
133             assert "ModuleNotFoundError" in msg
134
135
136     def test_healthcheck_no_wsl_binary():
137         r = _runner()
138         with patch.object(r, "_wsl_bash", side_effect=FileNotFoundError()):
139             ok, msg = r.healthcheck()
140             assert ok is False
141             assert "wsl" in msg.lower()
142
143
144     def test_run_predict_requires_weights():
145         r = TrackNetRunner(TrackNetConfig(weights_path="")) # no weights configured
146         assert r.run_predict(r"C:\v\clip.mp4", "out") is None
147
148
149     def test_run_predict_rejects_bad_extension(tmp_path):
150         r = _runner()
151         assert r.run_predict(r"C:\v\clip.mkv", str(tmp_path)) is None
152
153
154     def test_run_predict_success(tmp_path):
155         r = TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5", stage_in_wsl=False))
156         out_dir = str(tmp_path)
157         # predict.py would write "<stem>_predict.csv"; create it so the existence check
passes.
158         csv_path = os.path.join(out_dir, "clip_predict.csv")
159         with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="Prediction
complete.")):
160             open(csv_path, "w").close()
161             result = r.run_predict(r"C:\v\clip.mp4", out_dir)
162             assert result == csv_path

```

```

163
164
165 def test_run_predict_failure_returns_none(tmp_path):
166     r = TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5", stage_in_wsl=False))
167     with patch.object(r, "_wsl_bash", return_value=_proc(1, stderr="CUDA error")):
168         assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path)) is None
169
170
171 def test_run_predict_missing_output_csv(tmp_path):
172     """Exit 0 but no CSV produced should be treated as failure."""
173     r = TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5", stage_in_wsl=False))
174     with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="done")):
175         assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path)) is None
176
177
178 def test_stage_video_success():
179     r = _runner()
180     with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="OK")):
181         staged = r._stage_video(r"C:\v\clip.mp4", "clip.mp4")
182         assert staged == "~/clips/clip.mp4"
183
184
185 def test_stage_video_failure():
186     r = _runner()
187     with patch.object(r, "_wsl_bash", return_value=_proc(1, stderr="No such file")):
188         assert r._stage_video(r"C:\v\clip.mp4", "clip.mp4") is None
189
190
191 def test_run_predict_stages_when_enabled(tmp_path):
192     """With stage_in_wsl=True, run_predict should stage the video first."""
193     r = TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5", stage_in_wsl=True))
194     out_dir = str(tmp_path)
195     csv_path = os.path.join(out_dir, "clip_predict.csv")
196     with patch.object(r, "_stage_video", return_value="~/clips/clip.mp4") as mock_stage,
197         \
198         patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="done")):
199         open(csv_path, "w").close()
200         result = r.run_predict(r"C:\v\clip.mp4", out_dir)
201         mock_stage.assert_called_once()
202         assert result == csv_path
203
204 # ----- DetectorRunner ABC
contract
205
206 def test_tracknet_runner_implements_detector_runner_abc():
207     """TrackNetRunner must be a DetectorRunner (for polymorphism / future native
runners)."""
208     r = _runner()
209     assert isinstance(r, DetectorRunner)
210     # healthcheck contract: (bool, str) ? exercised with patch in other tests, but shape
here too
211     with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="TF ok")):
212         res = r.healthcheck()
213         assert isinstance(res, tuple) and len(res) == 2
214         assert isinstance(res[0], bool)
215         assert isinstance(res[1], str)
216
217
218 def test_wasb_runner_implements_detector_runner_abc():
219     """WasbRunner must also satisfy the DetectorRunner contract."""
220     w = WasbRunner(WasbConfig())
221     assert isinstance(w, DetectorRunner)
222     with patch.object(w, "_wsl_bash", return_value=_proc(0, stdout="torch ok")):
223         res = w.healthcheck()

```

```

224         assert isinstance(res, tuple) and len(res) == 2
225         assert isinstance(res[0], bool) and isinstance(res[1], str)
226
227
228     # (Intentionally no "no-wsl unpatched predict" test here: the stage/inference paths
229     # raise FileNotFoundError on this Linux box when "wsl" is absent, while only the
230     # healthcheck paths catch it today. The graceful-None cases are already exercised
231     # by the patched tests above and by the hybrid tests. Adding a broad except in
232     # run_predict would be a separate robustness change.)
233

```

40. user (2026-06-10T06:02:55.920Z)

```

1     import pytest
2     from unittest.mock import MagicMock, patch
3     from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
4
5     @patch("backend.pipeline.segmenters.gemini.provider_registry")
6     @patch("backend.pipeline.segmenters.gemini.get_video_duration")
7     @patch("os.environ.get")
8     def test_gemini_segmenter_single_file(mock_env_get, mock_get_dur,
mock_provider_registry):
9         mock_env_get.return_value = "dummy_api_key"
10        mock_get_dur.return_value = 100.0 # 100s video
11
12        mock_provider = MagicMock()
13        mock_provider.analyze_video.return_value = ([
14            {"start_time": 10.0, "end_time": 20.0, "shuttle_exchange_count": 5}
15        ], {"upload_time": 1, "process_time": 1, "gen_time": 1})
16        mock_provider_registry.get.return_value = mock_provider
17
18        db_mock = MagicMock()
19        db_mock.add_play_segment.return_value = "segment_1"
20
21        config = {
22            "indexing": {
23                "chunk_duration": 300, # Large chunk duration, falls back to single file
24                "chunk_overlap": 30
25            }
26        }
27
28        segmenter = GeminiPlaySegmenter(db_mock, config)
29        res = segmenter.process_video("v1", "dummy.mp4")
30
31        assert len(res) == 1
32        assert res[0]["id"] == "segment_1"
33        assert res[0]["start_time"] == 10.0
34        assert res[0]["end_time"] == 20.0
35        assert res[0]["metadata"]["shot_count"] == 5
36
37        @patch("backend.pipeline.segmenters.gemini.provider_registry")
38        @patch("backend.pipeline.segmenters.gemini.get_video_duration")
39        @patch("backend.pipeline.segmenters.gemini.extract_video_chunk")
40        @patch("os.environ.get")
41        def test_gemini_segmenter_chunked(mock_env_get, mock_extract, mock_get_dur,
mock_provider_registry):
42            mock_env_get.return_value = "dummy_api_key"
43            mock_get_dur.return_value = 200.0 # 200s video
44            mock_extract.return_value = True
45
46            mock_provider = MagicMock()
47            # Chunk 1 returns 1 segment, Chunk 2 returns another
48            def analyze_video_side_effect(path, prompt, chunk_duration, label, **kwargs):
49                if "Chunk 1" in label:
50                    return [{"start_time": 10.0, "end_time": 20.0}], {}

```


43. user (2026-06-10T06:03:49.754Z)

```
..... [ 28%]
..... [ 56%]
..... [ 84%]
..... [100%]
===== slowest 8 durations =====
8.51s call     tests/test_import_smoke.py::test_backend_main_imports
6.00s call     tests/test_provider_gemini.py::test_gemini_provider_upload_fail
0.32s call     tests/test_rally_seq_proto.py::test_signal_is_separable
0.29s call     tests/test_import_smoke.py::test_backend_package_has_no_syntax_errors
0.05s call     tests/test_database.py::test_candidate_link_and_unlabeled_filter
0.05s call     tests/test_eval_harness.py::test_cli_full_run
0.05s setup    tests/test_eval_regression.py::test_cli_update_baseline_then_clean_run
0.05s setup    tests/test_eval_regression.py::test_no_baseline_is_not_a_regression
256 passed in 25.85s
```

44. user (2026-06-10T06:03:59.123Z)

```
49:         # 1. Upload (retry transient network failures, e.g. WinError 10053 / reset
connections)
53:         for attempt in range(3):
62:             time.sleep(2 * (attempt + 1))
72:             time.sleep(10)
92:         # 3. Call Gemini (retry transient 503/UNAVAILABLE model-overload + network blips)
96:         for attempt in range(3):
113:            time.sleep(3 * (attempt + 1))
===
@app.get("/api/config")
def get_config():
    return global_config

@app.post("/api/process")
def process_video(req: ProcessRequest):
    if not os.path.exists(req.video_path):
        raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")

    video_id = compute_video_id(req.video_path)
    was_reingest = db_instance.get_video(video_id) is not None

    # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the report)
    logger.info(f"Running validations for {req.video_path}...")
    val_results, val_context = registry.run_all_with_context(req.video_path, global_config)
    failures = [r for r in val_results if not r.passed]

    # typed AppConfig: attribute access for the default segmenter (with safe fallback)
    default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
    seg_name = req.segmenter or default_seg
    segmenter_actual = None
    segmentation_ran = False
    response: Optional[Dict[str, Any]] = None
    try:
        if failures:
            db_instance.add_video(video_id, req.video_path, "failed", [r.dict() for r in
val_results])
            response = {
                "video_id": video_id,
                "status": "failed",
                "message": "Video failed validation checks.",
                "results": [r.dict() for r in val_results],
            }
    return response
```



```

# 2. Run segmenter & indexer
print(f"[API] Video passed checks. Segmenting and indexing...")
db_instance.add_video(video_id, req.video_path, "processed", [r.dict() for r in
val_results])

segmenter_cls = segmenter_registry.get(seg_name)
if not segmenter_cls:
    available = list(segmenter_registry.list_available().keys())
    raise HTTPException(
        status_code=400,
        detail=f"Segmenter '{seg_name}' not found. Available segmenters: {available}"
    )

print(f"[API] Using segmenter: {seg_name}")
segmenter_actual = seg_name
segmenter = segmenter_cls(db_instance, global_config)
result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
segmentation_ran = True

if isinstance(result, Failure):
    print(f"[API] Segmenter failed: {result.message}")
    response = {
        "video_id": video_id,
        "status": "failed",
        "message": f"Segmenter '{seg_name}' failed: {result.message}",
        "results": [r.dict() for r in val_results],
        "play_segments": [],
    }
    return response

segments = result.value if hasattr(result, "value") else result

response = {
    "video_id": video_id,
    "status": "success",
    "message": f"Successfully indexed {len(segments)} play segments using '{seg_name}'
segmenter.",
    "results": [r.dict() for r in val_results],
    "play_segments": segments,
}
return response
finally:
    # Always emit the per-video observability report (next to the video, or to
    # report.output_dir). Never raises. Surface the paths in the response.
    paths = emit_indexer_report(
        db=db_instance, video_id=video_id, video_path=req.video_path, config=global_config,
        val_results=val_results, val_context=val_context,
        segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
        was_reingest=was_reingest, segmentation_ran=segmentation_ran,
    )
    if paths and isinstance(response, dict):
        response["reports"] = paths

@app.post("/api/query")
def query_play_segments(req: QueryRequest):
    filters = {
        "server": req.server,
        "receiver": req.receiver,
        "duration_min": req.duration_min,
        "event_type": req.event_type
    }
    segments = db_instance.query_play_segments(req.video_id, filters)
    return {"play_segments": segments}

```

```

@app.post("/api/compile")
def compile_highlights(req: CompileRequest):
    video = db_instance.get_video(req.video_id)
    if not video:
        raise HTTPException(status_code=404, detail="Indexed video record not found.")

    # Retrieve matching play segments from db
    all_segments = db_instance.query_play_segments(req.video_id, {})
    matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]

    if not matched_segments:
        raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")

    # Extract start/end time pairs
    time_pairs = [(s["start_time"], s["end_time"]) for s in matched_segments]

    # Run compiler
    output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output") or
"output"
    compiler = VideoCompiler(output_dir)
    try:
        out_path = compiler.compile_highlights(video["filepath"], time_pairs,
req.output_filename)
        return {"status": "success", "filepath": out_path}
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Highlight stitching failed: {str(e)}")

```

--- CLI Debug Utility & Orchestration ---

```

def run_cli_diagnose_clip(video_path: str, timestamp: float):
    """CLI debugging utility to examine segment properties around a timestamp."""

```

45. user (2026-06-10T06:03:59.322Z)

```

1      """Tests for the indexer observability harvest + footgun rules.
2
3      These exercise the broken-run scenarios the report is meant to make debuggable.
4      obsreport is a soft dependency; skip cleanly if it isn't installed.
5      """
6      import os
7
8      import pytest
9
10     pytest.importorskip("obsreport")
11
12     from backend.reporting import emit_indexer_report, harvest # noqa: E402
13     from backend.pipeline.validators.base import ValidationResult # noqa: E402
14
15
16     class _FakeDB:
17         def __init__(self, play=None, candidates=None, existing=False):
18             self._play = play or []
19             self._candidates = candidates or []
20             self._existing = existing
21
22         def query_play_segments(self, video_id, filters):
23             return list(self._play)
24
25         def query_candidate_segments(self, video_id, detector=None, only_unlabeled=False):
26             return list(self._candidates)
27
28         def get_video(self, video_id):
29             return {"id": video_id} if self._existing else None
30

```

```

31
32     def _vr(name, passed, message="ok"):
33         return ValidationResult(validator_name=name, passed=passed, message=message)
34
35
36     _FFPROBE_OK = {
37         "streams": [{"width": 1920, "height": 1080, "r_frame_rate": "60000/1001",
"codec_name": "hevc"}],
38         "format": {"format_name": "mov,mp4,m4a", "duration": "676.0", "size": "55000000000"},
39     }
40
41
42     def _record(**kw):
43         base = dict(
44             video_id="md5abc",
45             video_path=None,
46             config={"ai_provider": "gemini", "ai_model": "gemini-flash", "indexing":
{"default_segmenter": "wasb_hybrid"}},
47             val_results=[_vr("format_check", True), _vr("relevance_check", True)],
48             val_context={"ffprobe_meta": _FFPROBE_OK},
49             play_segments=[{"duration": 5.0, "metadata": {"detector": "wasb-hybrid-
gemini"}}],
50             candidates=[{"id": 1}, {"id": 2}],
51             segmenter_requested="wasb_hybrid",
52             segmenter_actual="wasb_hybrid",
53             was_reingest=False,
54             segmentation_ran=True,
55         )
56         base.update(kw)
57         return harvest.record_run(**base).envelope
58
59
60     def _codes(env):
61         return {f.code for f in env.flags}
62
63
64     def test_healthy_run_is_clean(monkeypatch):
65         monkeypatch.setenv("GEMINI_API_KEY", "x")
66         env = _record()
67         assert env.verdict == "pass"
68         assert _codes(env) == set()
69         assert env.video_meta.fps == 59.94
70         assert env.video_meta.fps_source == "probed"
71         seg = env.get_stage("segmentation")
72         assert {m.key: m.value for m in seg.metrics}["segment_count"] == 1
73         assert {m.key: m.value for m in seg.metrics}["candidate_windows"] == 2
74         assert env.lint() == []
75
76
77     def test_silent_provider_noop_fires_when_no_key_and_zero_segments(monkeypatch):
78         monkeypatch.delenv("GEMINI_API_KEY", raising=False)
79         env = _record(play_segments=[])
80         assert "silent_provider_noop" in _codes(env)
81         assert env.verdict == "fail"
82         flag = next(f for f in env.flags if f.code == "silent_provider_noop")
83         assert flag.faq_ref == "README#Q7"
84
85
86     def test_ai_empty_warns_when_key_present_but_zero_segments(monkeypatch):
87         monkeypatch.setenv("GEMINI_API_KEY", "x")
88         env = _record(play_segments=[])
89         codes = _codes(env)
90         assert "ai_empty_vs_no_rallies" in codes
91         assert "silent_provider_noop" not in codes
92         assert env.verdict == "pass_with_warnings"

```

```

93
94
95 def test_motion_segmenter_flags_synthetic_metadata(monkeypatch):
96     monkeypatch.delenv("GEMINI_API_KEY", raising=False)
97     env = _record(segmenter_requested="motion", segmenter_actual="motion",
98                   config={"indexing": {"default_segmenter": "motion"}})
99     codes = _codes(env)
100     assert "synthetic_motion_metadata" in codes
101     # motion is not AI-based -> the provider-noop rule must NOT fire even with no key
102     assert "silent_provider_noop" not in codes
103
104
105 def test_segmenter_divergence_fires_only_without_explicit_override(monkeypatch):
106     monkeypatch.setenv("GEMINI_API_KEY", "x")
107     cfg = {"ai_provider": "gemini", "indexing": {"default_segmenter": "wasb_hybrid"}}
108     # No explicit request, but a different segmenter ran -> a fallback diverged: flag
109     it.
110     env = _record(segmenter_requested=None, segmenter_actual="gemini", config=cfg)
111     assert "segmenter_divergence" in _codes(env)
112     # Explicit --segmenter override that differs from config default is NOT a
113     divergence.
114     env2 = _record(segmenter_requested="gemini", segmenter_actual="gemini", config=cfg)
115     assert "segmenter_divergence" not in _codes(env2)
116
117
118 def test_reingest_info_flag(monkeypatch):
119     monkeypatch.setenv("GEMINI_API_KEY", "x")
120     env = _record(was_reingest=True)
121     assert "reingest_replaced_prior" in _codes(env)
122
123
124 def test_validation_failure_halts_and_flags(monkeypatch):
125     monkeypatch.setenv("GEMINI_API_KEY", "x")
126     env = _record(
127         val_results=[_vr("format_check", True), _vr("relevance_check", False, "court
128         color not found")],
129         play_segments=[],
130         segmenter_actual=None,
131         segmentation_ran=False,
132     )
133     assert env.get_stage("validation").status == "error"
134     assert env.get_stage("segmentation").status == "not_run"
135     assert "validation_failed" in _codes(env)
136     assert env.verdict == "fail"
137
138
139 def test_fps_fallback_flag_when_no_probe_but_segmented(monkeypatch):
140     monkeypatch.setenv("GEMINI_API_KEY", "x")
141     env = _record(val_context={}, segmentation_ran=True)
142     assert env.video_meta.fps_source == "fallback"
143     assert "fps_fallback_used" in _codes(env)
144
145
146 def test_emit_writes_files_next_to_video(tmp_path, monkeypatch):
147     monkeypatch.setenv("GEMINI_API_KEY", "x")
148     video = tmp_path / "match.mp4"
149     video.write_bytes(b"not really a video")
150     db = _FakeDB(play=[{"duration": 4.0, "metadata": {"detector": "wasb-hybrid"}}],
151                  candidates=[{"id": 1}])
152     paths = emit_indexer_report(
153         db=db, video_id="md5abc", video_path=str(video),
154         config={"ai_provider": "gemini", "indexing": {"default_segmenter":
155         "wasb_hybrid"}}},
156         val_results=[_vr("format_check", True)], val_context={"ffprobe_meta":
157         _FFPROBE_OK},

```

```

152         segmenter_requested="wasb_hybrid", segmenter_actual="wasb_hybrid",
153         was_reingest=False, segmentation_ran=True,
154     )
155     assert paths is not None
156     assert os.path.exists(paths["json"])
157     assert (tmp_path / "match.report.md").exists()
158     assert (tmp_path / "match.summary.md").exists()
159
160
161     def test_emit_accepts_typed_appconfig(tmp_path, monkeypatch):
162         """Reporting must work whether main passes a plain dict or the typed AppConfig
163         (backend.config) ? the reporting layer normalizes AppConfig -> dict."""
164         from backend.config import AppConfig
165
166         monkeypatch.setenv("GEMINI_API_KEY", "x")
167         video = tmp_path / "m.mp4"
168         video.write_bytes(b"x")
169         db = _FakeDB(play=[{"duration": 4.0, "metadata": {"detector": "wasb-hybrid"}}],
candidates=[{"id": 1}])
170         cfg = AppConfig() # typed config object, not a dict
171         paths = emit_indexer_report(
172             db=db, video_id="md5cfg", video_path=str(video), config=cfg,
173             val_results=[_vr("format_check", True)], val_context={"ffprobe_meta":
_FFPROBE_OK},
174             segmenter_requested="wasb_hybrid", segmenter_actual="wasb_hybrid",
175             was_reingest=False, segmentation_ran=True,
176         )
177         assert paths is not None
178         assert os.path.exists(paths["json"])
179
180
181     def test_emit_disabled_via_config(tmp_path, monkeypatch):
182         monkeypatch.setenv("GEMINI_API_KEY", "x")
183         db = _FakeDB()
184         paths = emit_indexer_report(
185             db=db, video_id="md5abc", video_path=str(tmp_path / "m.mp4"),
186             config={"report": {"enabled": False}},
187             val_results=[_vr("format_check", True)], val_context={},
188             segmenter_actual="motion", segmentation_ran=True,
189         )
190         assert paths is None
191

```

46. user (2026-06-10T06:04:18.474Z)

```

run_all_tests.py
run_eval.py
run_phase3_eval.py
run_process_and_stitch.py
run_regression.py
---
tests/test_eval_harness.py
tests/test_eval_regression.py
---
14:     def _get_connection(self):
15:         conn = sqlite3.connect(self.db_path)
23:         with self._get_connection() as conn:
105:             with self._get_connection() as conn:
118:             with self._get_connection() as conn:
132:             with self._get_connection() as conn:
150:             with self._get_connection() as conn:
169:             with self._get_connection() as conn:
196:         with self._get_connection() as conn:
206:         with self._get_connection() as conn:
243:         with self._get_connection() as conn:

```

265: with self._get_connection() as conn:

47. assistant (2026-06-10T06:05:19.425Z)

I have everything needed. Compiling the audit.

48. user (2026-06-10T06:06:36.353Z)

Structured output provided successfully

49. assistant (2026-06-10T06:06:42.018Z)

Audit complete. Structured output delivered: 256 tests across 39 files, all green in ~21s, 68% measured coverage, CI present ? with the critical gap being the untested FastAPI serving layer and ffmpeg stitcher, plus the untested wasb_hybrid twin where copy-paste drift would ship undetected.